

PRINT-FLOW-360 · RESEARCH LIBRARY

Qa Launch

9 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1.	Platform Audit - 2026-05-31
2.	Platform Gap Assessment - Print-Flow-360
3.	QA Findings - Manual Test Pass (2026-06-01)
4.	Admin (nuxt/ :3000) - Full Page QA Coverage - 2026-06-06
5.	Feedback Capture Readiness
6.	Critical money-path QA (signup → product config → cart → checkout → payment → order → fulfillment)
7.	Onboarding & first-run readiness for a brand-new store owner
8.	Support / Help-Desk Readiness
9.	Real-User QA, Onboarding & Support Readiness — Print-Flow-360

Platform Audit - 2026-05-31

Source: readme/research/qa-launch/PLATFORM_AUDIT_2026-05-31.md

IN SHORT — KEY TAKEAWAYS

This document is a health check of the whole platform — the admin area, the customer-facing store, and the backend code — done on 31 May 2026. It lists everything that was wrong, explains how bad each problem is, and records which fixes are already done. It matters because some of these problems directly affect what store owners and their customers see and experience.

The main parts explained simply:

- **False alarms** — Two things that looked like serious problems turned out to be fine. The money-formatting tool works correctly without extra setup, and no real passwords or secrets were ever saved into the public code. No action needed on these.
- **Currency display (the biggest problem, now fixed)** — The platform was always showing a dollar sign (“\$”) everywhere, even for stores that use rupees, euros, or other currencies. Orders, invoices, and quotes were also not saving which currency was used at the time, so if a store changed its currency later, old records could show the wrong one. All of this has been fixed and tested.
- **Confusing words shown to store owners (fixed)** — Order and invoice lists were showing internal code words like “open”, “draft”, and “void” instead of plain labels like “Open” or “Cancelled”. A customer’s order detail page also showed a long random ID string instead of something like “Item #1”. Both are fixed.
- **Silent failures (fixed)** — When a list page (orders, invoices, customers, etc.) failed to load data, the screen just stayed blank or kept spinning forever with no message. Now these pages show a clear error message and a “Retry” button.
- **“Coming soon” placeholders in live pages (fixed)** — Two tabs in the admin area (Payment Methods, Roles) were visible to store owners but did nothing. They have been removed so the interface no longer has dead ends.
- **Security hole in user roles (fixed)** — A bug allowed any logged-in user to read the roles and account names of OTHER stores, not just their own. This was a serious privacy issue. It is now fixed and tested — a regular user can only see their own store’s information.
- **Technical debt (tracked, not yet fixed)** — About 128 places in the admin app fetch data in a way that doesn’t follow the project’s own rules. Some pages also had requests that could pull tens of thousands of records at once with no limit. These don’t break anything today but make the code harder to maintain. They are logged for gradual clean-up.

What to do with this:

The most important fixes (currency, security hole, blank error screens) are already done and tested. The remaining open item is a slow background clean-up of the technical debt — no urgent action is needed from the store owner side.

Findings from a cross-app review (admin `nuxt/`, storefront `frontstore/`, Laravel `app/`). Each item lists file:line, what’s wrong, and the fix. Verified items were confirmed by hand; “needs review” items still require a check before acting.

✓ False alarms ruled out (do NOT act on these)

- **“formatMoney crashes because it’s never imported”** - FALSE. Nuxt auto-imports `app/utils/`. Already-committed components use `formatMoney()` with no import line and work fine.
- **“.env with secrets committed to git”** - FALSE. Only `.env.example` is tracked; the real `.env` is gitignored.

● P1 - Currency flow (store-level currency not threaded through display/records)

STATUS 2026-05-31 - display-correct fix DONE & tested. Backend

`CurrencyHelper::settingsForStore()` / `defaultStoreId()` resolve currency from `store_settings.default_currency_id` (→ legacy `currency_symbol` / `currency` → \$). Wired into order/invoice/quote/payment list+detail responses (`currency_symbol` / `currency_code` / `currency_decimals` , additive). Admin `formatMoney(amount, { symbol: row.currency_symbol })` threaded in the 4 lists. GTM 'INR' replaced with `currencySettings.currency` in `[slug].vue` + `index.vue` ; hardcoded USD shipping formatter + INR free-shipping promo also fixed. Test: `tests/Feature/Currency/CurrencyHelperTest.php` (5 passing). **Correction:** the authoritative column is `store_settings.default_currency_id` , NOT `stores.default_currency_id` (the latter doesn't exist - an early relation pointing there was caught by the test before shipping). **Update 4 - currency snapshot DONE & tested (P1 fully complete).** Migration `2026_06_04_000001` adds `currency_code` / `currency_symbol` / `currency_decimals` to orders/quotes/invoices (nullable). Model `creating` hooks snapshot the currency at creation (order → its store; quote/invoice → tenant primary store) on every creation path. List/detail controllers now ?? -prefer the stored snapshot and only backfill null (old) rows from live resolution - so a document keeps its original currency even if the store later changes it. Test: `tests/Feature/Currency/CurrencySnapshotTest.php` (3 passing incl. immutability). Migration applied to dev + test DBs. **Update 2:** financial detail/payment pages (orders/invoices/quotes detail + payment) now thread the record's `currency_symbol` / `currency_decimals` . Added `useStoreCurrency` composable (resolves `store_settings.default_currency` → legacy → \$, SSR-safe) and wired the store-scoped product views (`store/products/List` , `menu-management/MultiProductSelector` + `PreviewDropdown`). **Update 3 - admin currency display COMPLETE.** Added `GET /locations/currency/default` (`CurrencyController::tenantDefault` → `CurrencyHelper`) + `useTenantCurrency` composable (SSR-safe) for views showing tenant-level pricing. Wired ALL remaining product/job pricing callers: product-editor `tabs/General` + `Pricing` (incl. raw \$ literals), `category/ProductsList` , `store/blocks/ProductGridBlock` , `admin/product/*` (CombinationCreatorModal/Form, RangePriceEditor, CombinationPricing), `products/[uuid]/*` (combinations create/quick-create, pricing-rules), `job/details` + `job/KanbanBoard` . SaaS billing callers (`subscription` / `plans` / `billing/*` / `setting/account/*`) intentionally stay platform \$. Typecheck clean. **Only remaining P1 item: the currency snapshot migration (history robustness).**

Store currency is correctly modeled (`store_settings.default_currency_id` → `currencies.symbol/code`), but it is not carried into admin formatting or snapshotted onto transactions.

1. Admin shows \$ for every store regardless of currency.

- `nuxt/app/utils/formatMoney.ts:17` defaults `symbol` to '\$' ; none of the ~20 callers pass a symbol.
- Callers: `nuxt/app/components/order/List.vue` , `invoice/List.vue` , `quote/List.vue` , `category/ProductsList.vue` , `store/products/List.vue` , `store/products/tabs/Pricing.vue` , `billing/GatewaySelectModal.vue` , etc.
- Fix: thread the store currency symbol into `formatMoney(value, { symbol })` from store settings.

2. GTM/analytics hardcodes currency. `frontstore/app/pages/[slug].vue:276` → `currency: 'INR'` . Replace with the store's actual currency from `useThemeSettings().currencySettings` .

3. Quote & Invoice models lack `store_id` and there is no currency snapshot on orders/quotes/invoices/items.

- `app/Models/Quote.php` / `2025_02_01_095645_create_quotes_table.php` - no `store_id` .
- `app/Models/Invoice.php` / `2025_02_06_182911_create_invoices_table.php` - no `store_id` .
- Fix: migration to add `store_id` + a `currency_code` snapshot column; populate from store context on creation.

4. Two currency lookup paths can disagree. Storefront reads `useThemeSettings().currencySettings` (correct on product page); admin reads nothing and falls back to \$.

● P2 - Flow / UX breakage

STATUS 2026-05-31 - DONE. (a) Added `statusLabel()` to `nuxt/app/utils/statusColor.ts` (void → Cancelled, followup → Follow-up, Title-case fallback); used in order/quote/invoice lists instead of raw `{{ row.status }}`. (b) Added error+retry states to order/invoice/quote/payment lists (and fixed a payment-list bug where `loading` never reset on error → infinite spinner). © `quote.vue` UUID → “Item #N”. (d) Removed the dead “Payment Methods”/“Roles” coming-soon tabs from `admin/tenants/[id]`. (e) `StripeElementsDriver` blank `<div/>` → informative “choose another method” message (matches AuthorizeNet stub). Deferred: block embedded payment render_mode at gateway config (backend). (f) Also extended the error+retry state to the remaining two silent lists - `customer/List.vue` and `email/List.vue` (email's catch also now resets `loading`, fixing the same infinite-spinner bug); verified `customers` renders clean.

- **Raw status keys shown to store owners** - `nuxt/app/components/order/List.vue:143`, `quote/List.vue:172`, `invoice/List.vue:182` render `{{ row.status }}` (“open”, “draft”, “void”). A human-label map already exists at `frontstore/app/pages/track-order.vue:371` - reuse it.
- **Silent error states** - list components `catch` → `console.error` only (e.g. `nuxt/app/components/order/List.vue`), so an API failure = permanent blank/loading screen. Add the mandatory error state with a Retry action.
- **UUID shown to users** - `nuxt/app/pages/quote.vue:40` prints `Configuration ID: {{ item.uuid }}`. Remove or replace with `Item #{{ index + 1 }}`.
- **“Coming soon” tabs in production** - `nuxt/app/pages/admin/tenants/[id]/index.vue:117,129` (Payment Methods, Roles). Hide or gate behind a flag.
- **Dead-end payment drivers** - `frontstore/app/components/checkout/payment-drivers/AuthorizeNetEmbeddedDriver.vue` and `StripeElementsDriver.vue` are stubs. Block embedded mode at config time so checkout never dead-ends.

Already handled well: cart/checkout empty states, payment-gateway-not-configured warning, product page skeleton/error.

● P3 - Convention violations (tech debt, not crashes)

STATUS 2026-05-31 - safe items DONE. Localhost fallbacks: `useStorefrontUrl` / `useDesignerLink` / `chat.ts` already DEV-gated/defaulted; fixed the last two raw `apiBase || 'http://localhost:8000'` in `useProductPreview.ts` + `inquiry/Edit.vue` to fall back only when `environment === 'DEV'` (never fabricate localhost in prod). **Unbounded per_page - deliberately NOT blind-capped.** All four (`job/KanbanBoard` 10000, `job/CalendarWidget` 10000, `inquiry/KanbanBoard` 1000, editor `/categories` 500) feed *all-items* UIs (board columns / calendar / category picker); lowering `per_page` would silently drop data - a worse violation of `CLAUDE.md`'s “no silent caps”. Proper fix is per-feature (date-range scoping, lazy columns, searchable picker) - tracked as follow-up. **Update:** backend enabler for the calendar fix is DONE + tested - `PrintJobController::index` now accepts additive `date_from` / `date_to` / `date_field` (whitelisted column, always-AND, no behavior change when absent); `tests/Feature/Job/PrintJobDateRangeTest.php` (2 passing). Frontend `CalendarWidget` adoption (`datesSet` → bounded range refetch) deferred - it's a fetch-lifecycle refactor (always-mount calendar + de-race the immediate watchers/isJobLoading guard) needing in-browser verification across all 4 views; see task notes. **\$fetch-in-components (~128)** - left as tracked debt (large mechanical refactor; needs a lint rule).

- **~128 components call \$fetch / useFetch directly** instead of via composables (e.g. `nuxt/app/components/customer/List.vue`, `invoice/Email.vue`, `setting/product/ShopRate.vue`). Worth a lint rule.

- **Hardcoded `http://localhost` fallbacks** - `nuxt/app/composables/useProductPreview.ts:22` , `useStorefrontUrl.ts:15` , `useDesignerLink.ts:21` , `stores/chat.ts:176` , `components/inquiry/Edit.vue:57` . Remove fallbacks; require runtime config.
- **Unbounded queries** - `per_page: 10000` (`nuxt/app/components/job/KanbanBoard.vue:489` , `job/CalendarWidget.vue:252`), `per_page: 1000` (`inquiry/KanbanBoard..vue:88`), `per_page: 500` (`store-management/.../editor.vue:1302`). Violates the no-unbounded-queries invariant.

● P4 - Security (needs review before acting)

- **`request()->tenant_id` from query params** - `app/Http/Controllers/Api/Category/CategoryController.php:37` , `Api/Customer/CustomerController.php:68` , and ~25 others. **REVIEWED 2026-05-31 → mostly safe.** `InitializeTenancy` sets tenancy from `Auth::user()->tenant_id` (not the request) and every model these controllers query uses `BelongsToTenant` , so the global scope limits results to the user's tenant even when a `tenant_id` is supplied (a forged id yields an empty set). The `->when(request()->tenant_id, ...)` pattern is redundant-but-safe on tenant (`api.php`) routes and the legitimate tenant picker on landlord (`admin-api.php`) routes.
 - ● **EXCEPTION - FIXED 2026-05-31:** `RolesAndPermissionsController::index` (route `GET /roles` in `api.php` , auth+tenant middleware only, no super-admin gate) called `withoutTenancy()` and filtered by client-supplied `tenant_ids` whenever `request()->type === 'campaign'` - a client-controlled flag. Any tenant user could read other tenants' roles + tenant names via `?type=campaign&tenant_ids=1,2,3` (cross-tenant IDOR + tenant enumeration). Fix: gated the cross-tenant branch on `$isSuperAdmin` (was only used to hide the `admin` role label); non-admins now fall to the tenant-scoped branch. **Tested:** `tests/Feature/Security/AccessControlTest.php` (MTS-02) - a non-admin gets an empty result, a system admin still sees cross-tenant roles; the negative test was confirmed to FAIL against the pre-fix code (leaked the other tenant's role + tenant name).
 - **Note (not fixed, likely intentional):** `JobStatusController` / `OrderStatusController` use `withoutGlobalScopes()` on status reference tables (no client `tenant_id`); low-sensitivity config data, probably shared-by-design - confirm intent.
- **Google Maps key injected client-side at build time** - `nuxt.config.ts:43` . Normal for the Maps JS API *only if* the key is HTTP-referrer-restricted in Google Cloud. Confirm the restriction exists.

Clean: DOMPurify on all `v-html` , `httpOnly+secure+sameSite` cookies for auth tokens, security headers on routes, PII excluded from checkout `sessionStorage`.

Suggested order of attack

1. P1 currency: (a) thread store symbol into `formatMoney` callers, (b) fix hardcoded `'INR'` GTM tag, © migration for `store_id` + `currency_code` snapshot on quotes/invoices.
2. P4 `tenant_id` route-group check (high-stakes, quick).
3. P2 status labels + error states (high user impact, low effort).
4. P3 cleanups as ongoing tech debt.

Platform Gap Assessment - Print-Flow-360

Source: [readme/research/qa-launch/PLATFORM_GAP_ASSESSMENT_2026-06-07.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a thorough check-up of the Print-Flow-360 platform — like a mechanic inspecting every part of a car before a long trip. It lists what works well, what is broken, and what is missing, then ranks the problems by how badly they hurt the business. Reading it tells you where to focus time and money first.

The main parts explained simply:

- **Overall health score** — The platform is about 80% ready as an online print shop (where customers order and pay) but only about 40% ready as a full print-production system (where your team makes and ships the order). It is trying to do both jobs, which is a big advantage — but also means the gaps are spread across more ground.
- **The biggest problem: the order cannot travel cleanly from “paid” to “shipped”** — Right now, three links in the chain are broken. There is no automatic check that a customer's file is good enough to print (right resolution, right colours), no way to ship part of a large order while the rest is still being made, and no connection to courier services like FedEx or UPS to generate tracking numbers. Until these three gaps are closed, a real print shop cannot run fully on this platform.
- **What is already working well (do not touch these)** — The pricing engine (seven different ways to price a product), payments (six payment gateways), security, the drag-and-drop website builder, the design studio, and the B2B business-account features are all above the level of most competitors.
- **Tier 1 — Revenue problems (fix first)** — Missing: real shipping rates from couriers, taxes that change by region or country (important for legal reasons), a file check before printing starts, the ability to ship part of an order, and stopping overselling when stock runs low.
- **Tier 2 — Running a real shop at volume** — Missing: a job schedule to plan which orders print when, a proper refund process with reasons and inventory reversal, the ability to outsource jobs to other printers, and a link to accounting software like QuickBooks or Xero.
- **Tier 3 — Reaching more customers** — Missing: support for languages other than English, multiple currencies, a customer support/chat system, and easy sign-in with Google or Facebook.
- **Cheap wins (code is already written, just switched off)** — Several pricing types and the B2B department features are fully coded in the system but simply not turned on yet. These can be enabled quickly without building anything new.
- **Two things that looked broken but are actually fine** — The tax system does work (it just only supports one tax rate per store, not region-by-region). New store trial accounts also activate correctly — old notes suggested they did not, but the code was replaced and works.

What to do with this:

Fix the three broken links in the order chain first — file checking, partial shipment, and courier/tracking connection — as one joined project. That single epic makes the platform whole for real print shops. After that, add correct regional taxes and a refund process, then turn on the features that are already coded but switched off.

Senior-consultant review of what the platform is missing, benchmarked against the print-software market. Date: 2026-06-07 · Method: source-of-truth code review (not docs), four parallel domain sweeps, key claims verified directly.

Companion to: [readme/PRICING_REALWORLD_GAPS.md](#) , [readme/PLATFORM_AUDIT_2026-05-31.md](#) , [readme/FEATURE_IMPROVEMENT_RD_2026-06-02.md](#) .

0. TL;DR

Print-Flow-360 is ~**80% complete as a web-to-print storefront** and ~**40% complete as a print MIS / production system**. It straddles two markets most competitors pick one of. That breadth is the strategic edge - and the reason the gaps are spread thin across a wide surface.

The single most important finding: **the order-to-production spine breaks in three places** - there is no preflight/print-ready validation, no partial fulfillment, and no carrier/tracking integration. An order cannot travel cleanly from “paid” to “shipped.” Everything else is secondary to closing that chain.

Two findings initially flagged as “launch blockers” were **false alarms** caused by misreading commented/legacy code, and have been corrected here (see §6).

1. Method & how to read this

- Four read-only research sweeps over `app/` , `database/migrations/` , `nuxt/` , `frontstore/` , `designer/` , `pdf-service/` , and `readme/` .
- Benchmarked against the two halves of the print-software market:
 - **Web-to-print / storefront**: OnPrintShop, DesignNBuy, Inksoft, Gelato, Aleyant Pressero.
 - **Print MIS / production**: Printavo, Avanti, PrintVis, tFLOW.
- Every gap labelled **ABSENT** below was confirmed missing in code, not merely undocumented.
- Severity is ranked by **business impact**, not engineering size.

2. What’s strong (leave alone; polish only)

These are genuinely above market standard - do not refactor them chasing gaps:

Area	Why it’s strong
Pricing engine	7 wired strategies (fixed, %, area, quantity-tier, formula, conditional, combination-matrix), B2B contract/% overrides, frozen price snapshots on cart/quote/order. More capable than most competitors. <code>app/Services/Pricing/</code> .
Payments	Multi-gateway (Stripe, Razorpay, <code>Authorize.Net</code> , PayPal, PayTM, Cheque) via a registry/driver pattern; idempotent webhooks, per-gateway signature verification, dedup table, audit log. Production-grade. <code>readme/PAYMENT_GATEWAYS.md</code> .
Multi-tenancy & caching	<code>stanc1/tenancy</code> , <code>BelongsToTenant</code> , tenant-safe versioned cache with <code>x-tenant-host</code> vary. <code>readme/STOREFRONT_CACHE_GUIDE.md</code> .
Security	22-fix audit (May 2026): httpOnly cookie auth, DOMPurify <code>v-html</code> , open-redirect guard, CSP. <code>readme/SECURITY_GUIDELINES.md</code> .
CMS page builder	50+ content blocks, drag-drop, per-store theming, menu builder.
Design studio	Fabric.js canvas, templates with locked regions, personalization mode, IndexedDB crash recovery, QR/barcode, gated AI image/background tools.
B2B (further along than it looks)	<code>CompanyAccount</code> , <code>CreditAccount</code> + append-only <code>CreditLedgerEntry</code> , <code>CompanyPricingResolver</code> , credit-limit enforcement all exist in code. Just not surfaced in the storefront yet.

3. The gaps that matter - ranked by impact

● Tier 1 - Revenue leaks & market blockers (fix before scaling)

Gap	Business impact	Code evidence
No carrier / shipping integration	Can quote flat/tiered rates only. Cannot buy a label, pull a live UPS/FedEx/USPS/DHL rate, or push a real tracking number. "Track my order" is impossible. Daily expectation for every shop.	<code>app/Models/ShippingRate.php</code> is flat / free-threshold / per-item only. No carrier drivers, no shipment entity, no tracking poll. Email template with <code>{tracking_number}</code> exists but nothing populates it.
No destination / jurisdiction tax	A single flat rate per store. US sales-tax nexus, EU VAT, GST-by-region all unsupported - a compliance risk for tenants, not just a feature gap. No Avalara/TaxJar hook.	<code>StorefrontCheckoutController.php:360-393</code> - one <code>tax_rate</code> from store settings (engine is real, but single-rate).
No preflight / print-ready validation / CMYK	<i>The</i> print-specific gap. Files reach production with no resolution check, no bleed enforcement, no font-embed check, no CMYK conversion (output is RGB). Reprints and rejects come straight out of margin.	No preflight in <code>designer/</code> or <code>pdf-service/</code> ; PDF path does not color-convert.
No partial fulfillment / split shipments	An order is one atomic status. Cannot ship 500 of 1000, or split a multi-product order across shipments. Blocks large runs and B2B.	<code>Order</code> has a single <code>status</code> ; no per-item fulfilled counts, no shipment records.
No stock reservation at checkout	Inventory is not held/decremented when an order is placed → overselling under concurrency.	<code>ProductInventoryService</code> deducts manually; no cart → order hold.

● Tier 2 - Operational maturity (to run real shops at volume)

Gap	Why it matters	Evidence
No production scheduling / press queue	Imposition + press specs exist, but no scheduling engine: no job board, no capacity/sequencing, no operator assignment. The MIS half is half-built.	<code>app/Models/Imposition/Press.php</code> , <code>ImpositionService.php</code> exist; no scheduler, no operator role.
No refund / credit-memo / RMA workflow	<code>REFUNDED</code> status and <code>refunded_partial</code> exist but there is no refund service, no reason tracking, no inventory reversal, no return authorization.	<code>PaymentStatusEnum::REFUNDED_PARTIAL</code> unintegrated; no refund flow.
No vendor / trade-printing / outsourcing	No supplier model, no PO generation, no dropship. Most shops outsource something.	Absent.
No accounting sync (QuickBooks / Xero / Zoho Books)	Listed in the integration catalog, not implemented. Bookkeeping is manual export. High-frequency SMB ask.	Catalog entry only; no driver.
No raw-material / substrate inventory	Finished-product stock is tracked; paper/ink/substrate is not. Imposition computes material cost but it is not tied to stock.	<code>ProductInventoryService</code> is finished-goods only.
One-click reorder only implicit	Snapshots make it <i>possible</i> but there is no reorder flow. Reorders are the lifeblood of print (cards, recurring jobs).	No reorder endpoint/UI.

● Tier 3 - Growth & reach (new markets / less friction)

Gap	Impact
No i18n / multi-language storefront	Storefront is English-only (designer has partial i18n). Blocks non-English markets.
Multi-currency partial	Single currency per store, snapshotted; no selector, no FX, no per-country pricing.
No customer support layer	No helpdesk, ticketing, live chat, or knowledge base. Per-order message threads exist, but no support surface.
White-label custom domains partial	<code>Store.domain</code> exists; no DNS-setup UI, verification workflow, or SSL automation.
No PWA / offline; informal accessibility	No service worker/manifest; ~494 aria attributes but no WCAG audit - a procurement/legal risk for enterprise & government buyers.
SSO (Google/Facebook) scaffolded, not wired	Signup/login friction for B2C.

4. Cheap wins - code already exists, just unwired

Highest ROI-per-hour on the board. These are enum/UI work, not net-new builds:

- **Pricing strategies coded but not enabled:** `MultiplierPricingStrategy` , `PerUnitPricingStrategy` , `CharacterBasedPricingStrategy` , `HybridAreaQuantityPricingStrategy` (`app/Services/Pricing/Strategies/`).
- **Per-size base pricing:** `ProductSize.pricing_type/value/formula` columns exist; the calculator ignores them. Flagged Wave 1 in `readme/PRICING_REALWORLD_GAPS.md` .
- **B2B departments:** modeled (`app/Models/B2B/Department.php`), no management UI; B2B accounts not surfaced in storefront.
- **Imposition:** service written, not wired to the designer/order flow.

5. The strategic question

DECISION (2026-06-07): storefront-first. Implementation plan in `readme/STOREFRONT_FIRST_ROADMAP_2026-06-07.md` . Scouting found most of Tier 1 is dormant-but-built (EasyPost carrier driver, `runPreflight()` , CMYK `processImageForPrint()` all exist, just unwired) - the spine is largely wiring, not greenfield.

Are you a storefront that sells print, or the system that runs the print shop? Today you're ~80% of the first and ~40% of the second. You cannot credibly finish both at once.

- **Storefront-first** → prioritize shipping integration, destination tax, preflight/CMYK, reorder, i18n/currency. These convert and retain buyers directly.
- **MIS/production-first** → prioritize scheduling/press queue, partial fulfillment, materials inventory, vendor/outsourcing, accounting sync. These let a shop *operate* on you and drop their second tool.

Recommendation: finish the **storefront-to-production spine first** - `preflight` → `stock reservation` → `partial fulfillment` → `carrier + tracking` - as one continuous epic. That's the path a real order travels, it's where most investment already sits, and it's broken in exactly three places today. Closing it makes the product *whole* on the route you've already committed to, before widening scope.

6. Corrections to common misreads

Two items are frequently (and were initially) reported as broken. Both are **fine** - verified in code 2026-06-07:

Claimed "blocker"	Reality	Evidence
"Tax hardcoded to \$0 at checkout"	False. A real tax engine reads <code>tax_enabled</code> , <code>tax_rate</code> , <code>prices_include_tax</code> and handles inclusive/exclusive. The real gap is <i>single-rate</i> , not <i>zero</i> (see Tier 1).	<code>StorefrontCheckoutController.php:360-393</code> .
"Trial subscription auto-creation commented out"	False. Old inline code is commented, but replaced by <code>AccountStatusService::approveTrial(\$tenant, null)</code> on auto-approve, plus signup confirmation email + admin notification.	<code>AuthController.php:121-145</code> .

Lesson for the team: grep results showing commented blocks are not proof of absence - trace the live replacement before filing a blocker.

7. Suggested sequencing

1. **Spine repair (Tier 1), in order:** preflight/CMYK output → stock reservation → partial fulfillment → carrier integration + tracking. One epic.
2. **Money correctness:** destination tax (TaxJar/Avalara) → refund/credit-memo workflow.
3. **Cheap wins in parallel:** wire dormant pricing strategies + per-size pricing; surface B2B in the storefront.
4. **Reach:** i18n + multi-currency → accounting sync → reorder flow.
5. **Operational depth:** production scheduling → materials inventory → vendor/outsourcing.

8. Domain inventory (reference)

Condensed BUILT / PARTIAL / ABSENT map. Full per-domain notes available on request.

Product / Catalog / Pricing - BUILT: product types (hybrid/tshirt/packaging), sizes w/ bleed-trim guides, options + constraints, combination pricing, B2B pricing, product + size inventory, categories, reviews, quotes, configurations, Fabric.js designer + templates/assets. PARTIAL: per-size pricing (unwired), several strategy classes (unwired). ABSENT: preflight, proof-approval gate, design versioning, print-ready PDF export, graduated area pricing, rush surcharge, price rounding.

Order / Production / Fulfillment - BUILT: order lifecycle + custom statuses, immutable snapshots, order history audit, print jobs + custom job statuses, imposition + press specs, proofs w/ signature, multi-channel notifications, automation triggers, shipping rate calc. PARTIAL: refunds, production scheduling, operator assignment. ABSENT: carrier integration, label printing, tracking, partial fulfillment, vendor/dropship, stock reservation, raw-material inventory, RMA, multi-warehouse.

Business Ops (CRM / Reporting / Accounting / Billing / Integrations) - BUILT: customer profiles + segments + activity, B2B accounts + credit ledger, 20+ statistics reports, Action Center alerts, invoices/quotes w/ currency snapshot, multi-gateway payments + webhooks, SaaS subscription framework, integration registry (email/SMS/shipping/payment drivers), Zoho Sign, reCAPTCHA. PARTIAL: communication thread, usage metering, embedded card forms. ABSENT: accounting sync (QB/Xero), unified inbox, GL/expense tracking, usage-based billing, dunning, most catalog integrations (Zoho CRM, Mailchimp, Klaviyo).

Storefront / Designer / Infra - BUILT: catalog/search/cart/checkout, account/profile/addresses/wishlist, reviews, SEO (sitemap/robots/Google Shopping), themes + CMS, multi-tenancy, onboarding + seeding, caching, security. PARTIAL: reorder, white-label domains, multi-currency. ABSENT: i18n, PWA/offline, formal WCAG, helpdesk/live chat, SSO, CMYK output, preflight.

Prepared as a consulting deliverable. To convert any tier into a file-level implementation plan, start from the entry points cited above.

QA Findings - Manual Test Pass (2026-06-01)

Source: readme/research/qa-launch/QA_FINDINGS_2026-06-01.md

IN SHORT — KEY TAKEAWAYS

This document is a report from a test session where someone carefully walked through the Print-Flow-360 platform — the admin panel, the customer store, and the checkout — and wrote down every problem they found. Most of the problems have already been fixed. Reading this tells you what was broken, why it mattered, and what was done to repair it.

The main parts explained simply:

- **Cart price vs checkout price mismatch** — When a customer added items to their cart, the shipping cost shown in the cart was \$49. But the moment they clicked “Checkout,” the shipping jumped to \$100 with no explanation. This is a serious trust problem — a customer who sees a different price at checkout will abandon their order or dispute the charge later. This is still open and needs a fix.
- **Same product showing two different prices on one page** — A product listed as \$75 in one spot on its page showed \$77 in another spot on the same page. It looked broken and untrustworthy. This was fixed so both numbers now agree.
- **Too many requests error (429) under normal use** — The system was set to allow only 60 requests per minute per person. The admin dashboard alone fires about a dozen requests when it first loads. Open a couple of tabs and you’d hit the limit just doing normal work, causing blank screens. This was fixed by raising the limit for logged-in staff to 300 per minute.
- **Screens that showed “Loading...” forever or “No results” while data was actually loading** — Several pages would spin forever or show “0 records” even when the data was there and on its way. Fixed by separating the “loading” state from the “empty” state — now a blank result only shows after the data has fully arrived.
- **Team and Roles page was fully broken** — The page for managing staff roles showed “0 records” and a raw error message instead of the list. A bug deep in the server code caused this. Fixed — the team page now loads correctly.
- **Dashboard showing duplicate widgets** — The main dashboard showed the same Quotes and Invoices boxes twice. A layout bug caused this when widgets were moved around. Fixed so each widget appears only once.
- **Orders showing “Deleted customer” even when the customer’s details were saved** — When a customer account was removed, their name, email, and phone disappeared from all their past orders — even though the system had saved a copy of those details at the time the order was placed. Fixed so past orders now show the saved snapshot of customer details.
- **Security check: tenant data isolation** — A check was done to confirm that one store owner cannot see another store owner’s data. No leak was found. The system has three separate layers of protection.
- **Test suite was completely broken** — A typo in one test file stopped all automated tests from running. Fixed, so the safety net is back on.

What to do with this:

The one remaining open issue is the **shipping price mismatch between cart and checkout** — this will cause customer confusion and lost sales and should be fixed before real customers use the store. Everything else listed here has been resolved.

First exploratory QA pass against local dev (admin :3000, storefront :3001, API :8000). Goal: surface bugs before real users do. Tester: automated browser walkthrough. Status legend: ● blocker · ● high · ● medium · ● low/polish

1. Shipping cost & total change between Cart and Checkout (same items)

Where: Storefront (`printdesign.localhost:3001`) → Cart → Checkout **Steps:** Add Professional Letterhead (qty 1) + Vinyl Banners (qty 2) to cart → view cart → click Checkout.

Page	Subtotal	Shipping	Total
Cart (<code>/cart</code>)	\$108.65	\$49.00	\$157.65
Checkout (<code>/checkout?mode=order</code>)	\$108.65	\$100.00	\$208.65

Subtotal is identical, but **shipping silently jumps \$49 → \$100** and the total rises **\$51** with no explanation. This is the classic conversion-killer: the price the customer agreed to in the cart is not the price at payment.

Why it matters: Directly violates the price-calculator-sync rule (cart and checkout must agree). Erodes trust; will cause abandonment and chargebacks/disputes. **Likely cause:** Shipping is calculated differently on the cart page vs the checkout page (different default method, or checkout recomputes shipping without the cart's rule). Needs investigation in the shipping/estimate logic on both surfaces. **Fix direction:** Single source of truth for shipping - compute once (server-side) and carry the same value through cart → checkout. Add a test that asserts cart total === checkout total for the same cart.

2. Same product shows two different prices on one screen (\$77 vs \$75) - FIXED

Where: Storefront product page - Professional Letterhead. **Detail:** Header and Estimated Total show **\$77.00**; the “Recently Viewed” widget on the same page lists Professional Letterhead at **\$75.00**. **Why it matters:** Two prices for the same product on the same screen looks broken and untrustworthy. Likely a base-price vs from-price mismatch, or a stale cached price in the Recently Viewed component. **Fix direction:** Make the listing/recently-viewed price use the same “from” price source as the product page.

Root cause (`frontstore/app/components/product/ProductInfoPage.vue`): the header headline bound `formatted-price="store.totalPrice > 0 ? store.formattedPrice : formatBasePrice"` while keeping the “From” label (`show-starting-from="hasOptions"`). Once `calculate-price` returned, the header swapped from the base price (\$75) to the **configured total** (\$77 = base + the default-selected Paper Stock / Print Type options) but still read “From \$77” - duplicating the separate ESTIMATED TOTAL panel and contradicting the catalog listing + Recently Viewed, which both show the \$75 base. **Fix applied:** added a `headerPrice` computed - when the product has options, the header always shows `formatBasePrice` (the stable “From” starting price), so it stays \$75 and matches everywhere; the live configured price keeps its own ESTIMATED TOTAL panel. Products without options (no “From” concept) still show the resolved price. **Verified live:** header now reads “FROM \$75.00”, ESTIMATED TOTAL “\$77.00”, Recently Viewed “\$75.00” - all consistent. Pure display fix; calculation logic unchanged, so no admin mirror needed.

3. API returns 429 (Too Many Requests) under normal concurrent use - FIXED

Where: Admin - observed on `/business-accounts/:uuid` (and a transient `500` on `/store-management/stores/:uuid`).

Detail: With a few tabs open, page loads intermittently failed: one rendered the full-screen “**429 Too Many Requests**” Nuxt error page; on reload it loaded fine. A store page showed a bare “**500 internal server error**” then recovered on reload. **Why it matters:** Real usage = multiple tabs/users = concurrent requests. If the rate limiter trips this easily in single-user testing, it will trip constantly with real traffic. (Note: a second dev server was running during the test, which may have inflated load - but the limiter threshold still looks too low for normal app boot, which fires many parallel calls.) **Fix direction:** Review the API

rate-limit thresholds for authenticated admin traffic; the app's own boot/dashboard makes many parallel calls and should not self-trip. Exclude or raise limits for first-party authenticated requests.

Root cause: The `api` rate limiter in `AppServiceProvider` was a flat `Limit::perMinute(60)` for everyone. The admin dashboard boots ~a dozen widgets (each its own request) in parallel; with 2-3 tabs open a single admin exceeds 60/min in normal use and self-trips. **Fix applied** (`app/Providers/AppServiceProvider.php`): split the limit - an **authenticated** user (keyed by user id) now gets **300/min**, generous enough for the app's own parallel boot; **anonymous** traffic stays at **60/min per IP** to preserve abuse protection. If 300/min still proves tight under heavy multi-tab use, raise further or move the heaviest dashboard reads behind a single batched endpoint.

4. Frontend handles API failure with infinite skeletons / raw error pages (no retry) - **FIXED**

Where: Admin - business-account detail page (when the API 429'd). **Detail:** On the failed load, the page sat on **grey skeleton boxes forever** (never resolved to content or an error state). Separately, the store page showed a **raw "500 internal server error"** with no recovery action. **Why it matters:** Violates the mandatory loading/empty/error state rule and "never surface raw technical output." A skeleton that animates forever reads as "broken"; a bare 500 gives the user nowhere to go. **Fix direction:** Every data view needs a real error state with a **Retry** action and plain-language message. Skeletons must resolve to content or a proper empty/error state - never spin indefinitely.

Investigation + fix:

- Infinite skeletons (business-account detail):* the page already had a proper loading skeleton **and** an error card with a plain-language message + "Try again" retry, with `loading=false` in `finally`. The "forever" skeleton was the **in-flight 429** (request neither resolved nor failed while rate-limited), not a missing error state - resolved by the #3 rate-limit fix.
- Raw error / wrong empty state (store detail, `nuxt/app/pages/store-management/stores/[uuid]/index.vue`):* `fetchStore` had no error branch - on any failure (incl. 500, which the composable flattens to `{ok:false}`) it fell through to the **"Store not found"** empty state, falsely telling the owner the store doesn't exist, with only a "Back" link and no retry. Added an `errored` state + try/catch/finally and a distinct recoverable error card ("We couldn't load this store" + **Try again** + Back to stores), separate from the true not-found empty state.

5. Dashboard takes ~10s to fully load; long blank "Setting things up..." splash - **RESOLVED (verify)**

Where: Admin  (dashboard). **Detail:** Full-screen "Setting things up please wait..." splash for several seconds, then dashboard widgets each show "Loading metrics... / Loading jobs... / Loading leads..." for ~10s total before resolving. **Why it matters:** First impression of the whole product. Not broken, but slow enough to feel broken; the long blank splash is risky (looks hung). **Fix direction:** Render the dashboard shell immediately and let widgets fill in independently (they mostly do already). Investigate why the initial splash holds so long - likely the auth/bootstrap blocking on a slow or rate-limited call (see #3).

Re-verified live (after the #3 rate-limit fix): the dashboard shell now renders immediately and each widget fills in independently and resolves (Overview, Invoices, Quotes, Action Center, Job Board, Web Leads, Reports, metrics) - no long blank splash observed, no widgets stuck loading, **and no duplicates** (also confirms #8). The earlier ~10s/long-splash behaviour was driven by the boot's parallel calls self-tripping the 60/min limiter and retrying; with authenticated traffic now at 300/min that contention is gone. Keep an eye on it under heavier multi-tenant load.

6. Minor: stray “Hover Hover” text in Products table

Where: Admin `/products` - Options column, e.g. Bookmarks row. **Detail:** The text “Hover Hover” appears to leak into the cell (likely a tooltip label rendering inline instead of on hover). **Fix direction:** Check the options/variants tooltip in the products table cell.

7. Roles/Users (Team) page was completely broken - 500 + raw error leaked to UI - FIXED

Where: Admin → Settings → Team (`/setting/team`). Endpoint `GET /api/v1/roles`. **Symptom:** Roles and Users both showed “0 records”; a red toast appeared with the raw text “Class name must be a valid object or a string.” Tenant could not view or manage any roles/permissions.

Root cause (two compounding bugs):

- Config:** Sanctum registers a `sanctum` guard at runtime with `provider => null` (`config/auth.php` only defined `web/api/customer`). During a Sanctum-authenticated request, `sanctum` becomes the active default guard. Spatie's `Role::users()` morph relation (used by `withCount(['permissions', 'users'])` in `RolesAndPermissionsController::index`) resolves its model via `getModelForGuard(config('auth.defaults.guard'))` → `getModelForGuard('sanctum')` → `null` → `morphedByMany(null, ...)` → PHP `Error`: “Class name must be a valid object or a string.” (Reproduced deterministically: set default guard to `sanctum` then `Role::withCount('users')` throws; only happened over HTTP, not tinker, because tinker's default guard is `api`.)
- Error handling:** the controller's `catch (\Exception $e)` did not catch this - it's a PHP `\Error`, not `\Exception` - so it escaped as an unhandled 500 with the raw message instead of the intended “Failed to fetch roles.”

Fix applied:

- `config/auth.php` - added explicit `sanctum` guard with `provider => 'users'`.
- `RolesAndPermissionsController::index` - `catch (\Exception)` → `catch (\Throwable)` so model/relation `Error`s never leak raw to the UI.

Verified: Team page now loads 7 roles (Admin/Designer/Sales Rep/Project Manager/Manager/...); roles endpoint returns 200. **Follow-up to check:** any OTHER controller that does `catch (\Exception)` around model/relation resolution has the same raw-leak risk; and audit other Spatie/role queries that rely on the default guard.

8. Dashboard renders duplicate widgets - FIXED

Where: Admin dashboard `/`. **Detail:** “Quotes (4)” status widget appears **twice**; “Invoices (7)” table appears **twice**; the Invoices summary block repeats. Likely a default/seeded widget-layout with duplicates, or the widget list renders dupes. **Why it matters:** A brand-new tenant's first screen looks broken/redundant.

Root cause: `useDashboardLayout.mergeWithDefaults()` only scanned the left/right/bottom columns, **not** the nested `twoColumnGridWidgets`. When a user dragged the Invoice or Quote widget into the two-column grid, it left its original column; on the next layout-version merge the widget looked “missing” and was re-added to the column - so it rendered in both the grid and the column. **Fix applied (`nuxt/app/composables/useDashboardLayout.ts`):** (1) merge now treats ids present in any zone (grid included) as already-placed, so nothing gets re-added; (2) added `dedupeLayout()` in `applyLayout()` - a single id can only appear in the first zone it's found in, so layouts already saved with duplicates self-heal on next load and the clean version is persisted on the next save.

9. Account Settings preview shows wrong website URL - FIXED

Where: Admin → Settings → Account. The customer-facing “Preview” card links the website to `http://localhost:3000/register` (the admin register page) instead of the tenant's storefront URL.

Root cause: The Preview card bound `:href="businessFormData.website_url"` raw. When the stored value has no scheme (a non-technical owner typing `yourbusiness.com`, or a value like `localhost:3000/register`), the browser treats it as a relative path or a bogus custom scheme - so the link resolves against the admin origin (`localhost:3000`) and goes nowhere useful. **Fix applied** (`nuxt/app/components/account/Business/Form.vue`): added a `websiteHref` computed that prepends `https://` when the value lacks `http(s)://`, used in both the mobile and desktop preview cards (with `rel="noopener noreferrer"` since they open in a new tab). The displayed text still shows exactly what the owner typed.

10. Test suite won't run - syntax error blocks ALL PHPUnit tests - FIXED

Where: `tests/Feature/Storefront/CartEdgeCasesTest.php` (PHP reports line 162, but it's likely an invalid string interpolation elsewhere in the file). **Detail:** `vendor/bin/phpunit` (full suite, or any `--filter`) aborts with “syntax error, unexpected string content” before running anything, because PHPUnit parses every test file. Net effect: **the whole test suite is currently un-runnable** via the normal entrypoint. (Individual dirs can still be run by explicit path, e.g. `phpunit tests/Feature/Auth` - which passes 14/14.) **Why it matters:** CI/regression safety net is effectively down. Per the project's own “tests are required” rule, this should be fixed first so fixes can be verified. **Fix direction:** Repair the syntax in `CartEdgeCasesTest` (look for a double-quoted string with a quoted array key like `"$arr['key']"`, a smart-quote, or an unterminated heredoc).

Fix applied: A stray backtick (``) sat alone on line 129 - a PHP shell-exec string opener that was never closed, so the parser swallowed everything after it and misreported the failure at line 162. Removed it. The suite now loads **290 tests** (was 0 discoverable); `CartEdgeCasesTest` passes 4/4. Verified with `php -l` across all of `tests/` (all clean) and `phpunit --list-tests`.

11. Tenant isolation review - NO LEAK FOUND (verified)

Context: Part of the tenant-lifecycle pass - the critical security question is whether one tenant can read another's data. Triggered by noticing the admin sends `tenant_id` as a **query parameter** (e.g. `GET /api/v1/roles?...&tenant_id=<uuid>`), and that ~30 controllers filter with `->when(request()->tenant_id, fn($q) => $q->where('tenant_id', request()->tenant_id))` - i.e. they trust a client-supplied tenant id.

Why it's still safe (defense-in-depth, all three layers present):

1. Tenant context is derived server-side from the authenticated user, not the request.

`app/Http/Middleware/InitializeTenancy.php` calls `tenancy()->initialize(Auth::user()->tenant_id)` - the tenant comes from the logged-in user, so it can't be spoofed via query param or host header. This middleware wraps the whole `api/v1` group (`routes/api.php:83`).

2. `BelongsToTenant` global scope enforces it on every tenant-owned model. Spot-checked: User, Customer, Address, Tag, Note, NoteType, Task, TaskTemplate, Category, Role all `use BelongsToTenant`. With tenancy initialized to the auth tenant, every query gets an automatic `where tenant_id = <auth tenant>`.

3. The client-supplied `tenant_id` can only narrow, never widen. A spoofed `?tenant_id=<other tenant>` yields `tenant_id = mine AND tenant_id = theirs` → **empty**, never another tenant's rows. Cross-tenant reads (campaign targeting in `RolesAndPermissionsController`) are explicitly gated on `is_system_admin`, with a dev comment naming exactly this attack.

Residual notes (hygiene, not vulnerabilities):

- The admin frontend sending `tenant_id` in query strings is unnecessary and against the [CLAUDE.md](#) convention (“never pass tenant IDs in query params”). Harmless today, but worth removing so it can't become a foot-gun.
- The `where('tenant_id', request()->tenant_id)` pattern is a **latent** risk: it's only safe because the global scope is the real guard. If a future change adds `withoutTenancy()` or runs the query outside an authenticated/tenancy-initialized context, the client value would become the sole filter. Recommend dropping the client- `tenant_id` trust from these controllers entirely and relying on the global scope.
- Product **sub-entity** models (ProductOption, OptionGroup, PricingRule, ProductSize, etc.) don't use `BelongsToTenant` - they're scoped *through* their parent product (which is tenant-scoped). Fine as long as they're always loaded via the product and never queried directly by a client-supplied id; none of the audited controllers do.

12. “0 roles / 0 stores / 0 users” - NOT an onboarding gap (investigated)

Trigger: During tenant testing the admin showed a tenant (Brexis Wazik, `a1bb81a2`) with **0 roles**, **0 stores**, and **0 users** - looked like onboarding failed to seed.

Reality - the data is all there and loads fine:

- DB for this tenant: **7 roles** (admin/designer/sales-rep/project-manager/manager/prepress-manager/production-manager), **2 active stores** (Printdesign + B2B store; 7 more are soft-deleted), **1 user** (the owner-admin). Dashboard invoice/quote numbers (7 invoices / 4 quotes) matched this tenant exactly.
- The “0” displays were **transient**: (a) the #3 rate-limit (429) made list endpoints return empty, which the composables render as “0 records”; and (b) reading the screen before the async lists resolved. After the #3 fix, Roles shows **7** and Stores shows **All 2** live. Verified roles/users/stores queries return the right counts in tinker under this tenant, and confirmed the `/stores` endpoint resolves `tenant_id` correctly (debug-logged `count_for_tenant=2`, then removed the logging).

Genuine secondary findings surfaced by this dig:

1. Non-admin roles seed with ZERO permissions (by design, but an onboarding UX gap).

`TenantOnboardingService::seedRolesAndPermissions` creates roles and permissions but **never links them** (`role_has_permissions` is empty for every role in every tenant), and `config/permission.php` defines no role → permission mapping. The `admin` role still has full access via `Gate::before(fn($u) => $u->hasRole(['system-admin', 'admin'])) ?: null`. But **Designer / Sales Rep / Project Manager / etc. have no permissions at all** - assign a teammate one of those roles and they can access nothing, with no on-screen hint that the owner must configure the role first. Decision needed: seed sensible per-role defaults, or add UX guidance on the Roles screen (“This role has no permissions yet - assign some so members can work”).

2. List pages conflate failure and empty, with raw empty states. Team Users/Roles and Stores render “0 records / You’ve reached the end” both when truly empty *and* when the request fails (429/error). For a non-technical owner, “0 roles” reads as “my data is gone.” Lists need the same loading/empty/error discipline as detail pages: a helpful empty state (“You haven’t added team members yet - invite your team”) and a distinct error+retry, never a bare “0 records” on failure.

3. `routes/store-api.php` does not run `InitializeTenancy`. `tenant()` is null on all store-api routes; every store-api controller compensates with the `tenant()->id ?? auth()->user()->tenant_id` fallback (consistently, so it's not currently a bug). Worth aligning with `routes/api.php` (which wraps everything in `InitializeTenancy`) so `tenant()` is reliable and the per-controller fallbacks can be dropped.

13. Orders show “Deleted customer” even though a full customer snapshot was saved - FIXED

Where: Admin → Orders (`nuxt/app/components/order/List.vue` customer block, lines 235–254). **Detail:** All 28 orders for the test tenant render “Deleted customer / - / - / -”. The 2 customers behind those orders were soft-deleted (2026-05-31), so the live `customer` relation is null and the UI falls back to “Deleted customer”. *Technically* the null-guard is working - **but** every order also carries a populated `customer_snapshot` captured at order time:

```
{ "id": 6, "uuid": "...", "name": "Brexix Wazik", "email": "abhixx048@gmail.com", "phone": "1231231231", "company_name": "Company Name" }
```

The order list (and the order resource) **ignore the snapshot entirely** - `customer_snapshot` is not exposed by any API Resource and not referenced anywhere in `nuxt/app`. So a print shop that removes a customer instantly loses the name, email, and phone on **every past order** from them, even though the data was deliberately snapshotted to prevent exactly that.

Why it matters: Orders are financial records. “Who placed this \$540 order and how do I reach them?” must survive a customer being deleted. Showing “Deleted customer / -” when the snapshot has the name+email+phone is both worse UX and a fulfillment/dispute risk. **Fix direction:** (1) expose `customer_snapshot` in the order list/detail API resource; (2) in `List.vue` (and order detail), when `row.customer` is null, fall back to `row.customer_snapshot` (name / company_name / email / phone), ideally with a subtle “(deleted)” tag - never drop to a bare “Deleted customer / -” when the snapshot exists.

Minor order-list observations (same screen)

-  **Raw option identifiers leak** on some items: ORDER-#0023 “Custom Water Bottle” shows `Options: Option 55: 178, Option 56: 181, Option 57: 182...` - internal option/value IDs instead of human labels (other products show proper labels like “Paper Type: Glossy”). The product’s options were never named; the UI should not render `Option 55: 178`.
-  **“Product Name” placeholder** - ORDER-#0001 contains an item literally named “Product Name” (\$110) - seeded placeholder data leaking into a real order.
-  **Coupon code “NEW10” leaks** into the order header/timeline area (ORDER-#0008/#0009) between “Initiated” and the date.
-  **Every order is “OVERDUE”** with Due date == order date - likely a default due-date (no lead-time) issue or seed data.
-  **Placed orders labelled “ESTIMATED TOTAL”** - once an order exists the figure is the actual total, not an estimate.

14. List screens show “Loading...” and “No results” at the same time (and look empty for 3–5s) - FIXED (invoices/quotes)

Where: Admin list screens - confirmed on **Invoices** (`/invoices`), and the same pattern drives the transient “0 records” seen on Roles, Stores, and Orders during this pass. **Detail:** On first navigation the Invoices table renders “Loading...” and “No results” simultaneously, and the header reads “0 total”, for ~3–5 seconds - then resolves to all 7 invoices (verified the API returns `200` with `data: array(7)` / `meta.total: 7` the whole time; a manual **Refresh** also fixes it instantly). So the data is never missing - the list just renders its **empty state while it is still loading**. **Why it matters:** A store owner opening Invoices/Orders sees “No results / 0 total” and reasonably concludes they have none (or that data was lost). This is the list-screen version of the mandatory loading/empty/error rule: while loading, show **only** a skeleton/spinner - never the “No results” empty state - and only show empty once the request has resolved with zero rows. (Also: 3–5s to first paint of a 7-row list is slow; worth profiling the invoices list query/serialization.) **Note:** This is the same root cause behind several “0 roles / 0 stores / 0 records” observations earlier in this document - they were the empty state showing before the async list resolved (compounded earlier by the #3 429). Fixing the loading/empty precedence removes a whole class of “looks broken but isn’t” reports.

Minor invoices observations

- 5 of 7 invoices show “\$0.00” total (#0001–#0004, #0007) - likely empty drafts, but worth confirming the invoice total computes from line items rather than defaulting to 0.

✓ Flows that worked

- **Login / session** - already authenticated; dashboard reachable.
- **Dashboard data** - Invoices (\$2,570 total, 4 new), Quotes (4) rendered correctly once loaded.
- **Products list** (`/products`) - 47 products, stats cards, table all correct.
- **Business account detail** - Overview / Contacts / Departments / Catalog & Pricing / Account & Credit tabs render (when not rate-limited).
- **Add to cart** - storefront, with options and quantity, fires “Item added to cart” toast and updates cart badge.
- **Cart** - line items, quantities, per-unit price, subtotal/shipping/total all math-correct (subtotal \$108.65 ✓).
- **Checkout form** - clean 2-step flow; Place Order correctly disabled with a plain-language reason (“Please add a delivery address...”). (*Did not place a real order.*)

Not yet tested (next pass)

Orders list & detail · Quotes create/convert · Invoices & payment recording · Customers CRUD · Designer studio · Store management CMS/blocks · Settings (templates edit - recently changed) · Mobile (375px) on all of the above · the actual Place Order → payment → order-created → print-job path.

Admin (nuxt/ :3000) - Full Page QA Coverage - 2026-06-06

Source: readme/research/qa-launch/QA_PAGE_COVERAGE_2026-06-06.md

IN SHORT — KEY TAKEAWAYS

This document is a testing log from one session where every page of the store's admin panel and customer-facing shop was visited, clicked through, and checked for errors. It records what was broken, what got fixed on the spot, and what still needs attention. It matters because it shows the real state of the platform before going live with real customers and real money.

The main parts explained simply:

- **Pages visited and checked** — The tester opened every admin screen (customers, orders, products, settings, accounting, marketing, and more) and watched for errors in the background. Nearly all pages passed with no errors.
- **Fixes made during the session** — Four problems were found and fixed on the spot: error messages that crashed the activity log every time a newsletter was saved; a live-chat crash that was silently breaking every admin page; order item images that showed broken picture links; and a "Transactions" page that was an empty stub now redirected to the correct Accounting page.
- **Minor issues left open** — Three small cosmetic or data problems were noted but not yet fixed: a status label using an hourglass emoji while everything else uses a colored dot; a web inquiry that accidentally saved a form-validation message instead of the customer's real message; and an AI usage counter where about 70 requests do not appear in any summary bucket.
- **End-to-end flows tested** — The tester actually created a customer, a quote with a line item, an email campaign, and changed an order status — all the way through, with no backend errors. Audit logs (the record of who did what and when) were confirmed to be working.
- **Storefront pricing verified** — All five pricing methods (fixed price, area-based, quantity tiers, percentage, and formula) were tested on real product pages and produced correct prices.
- **Full purchase from shop to admin** — A guest customer added a product to the cart and checked out. The order appeared correctly in the admin panel with the right customer, product, and payment method.
- **Shipping cost bug (critical — fixed)** — When a customer checked out, the shop correctly showed a shipping charge (for example \$100) but the admin order recorded \$0 shipping, meaning the store was losing that revenue on every order. This was found, root-caused, and fixed. New orders now save the correct shipping cost. Coupon discounts were also not being subtracted from the order total — this was fixed at the same time. Tests were written to confirm the fix.
- **Zero-price item checkout bug (fixed)** — A product with no calculable price could still be added to the cart from an old session and slip through to checkout. The checkout page now blocks this and tells the customer to request a quote or remove the item.
- **Other small storefront issues (not yet fixed)** — The default payment method shown to customers is one that doesn't work yet ([Authorize.Net](#)); a better default like Cheque or Stripe should be set. The order success page tells a customer "Payment Successful" even for cheque orders where payment hasn't actually been received yet — the wording should say "Order placed, payment pending." Removing an item from the cart updates totals immediately but the item card stays visible until the page is refreshed.

What to do with this:

The shipping + coupon fix is the most important one — it was causing real revenue loss and is now resolved. Before launch, also fix the default payment method so the first thing customers see actually works, and update the cheque success page message so it doesn't falsely promise that payment was received.

Goal: reach **every** admin page, enter/submit data where applicable, watch console + Laravel logs, and fix anything broken or sub-par along the way. Status legend:

-  not yet visited

-  visited, no errors
-  visited, minor issue (noted)
-  visited, broken / error (noted)

Dynamic routes (`[id]` , `[uuid]` , `[slug]`) are tested by visiting one real record reachable from the corresponding list page.

Fixes applied this session

-  `config/logging.php` - added missing `audit` channel (was throwing EMERGENCY on every email-campaign / newsletter / segment write).
-  `chat.ts` socket failure no longer bubbles to the global Vue error handler - `FloatingChat.vue` + `Panel.vue` now catch a failed `initSocketConnection()` and degrade gracefully.
-  **Broken order-item images (Orders list + detail)** - designer order items stored a *double-encoded* JSON blob in `items.image_url ( {"image":{"...inner json..."}} )`, so the `image_url_thumbnail_url` accessor produced `s3.com/<url-encoded-json>` → 404. Fixed `FileHelper::parseImagePaths()` to unwrap a nested JSON path value (`flattenImagePath` , prefers thumbnail → image → first page); fixed `orders/[id].vue` to use the accessor instead of binding raw `item.image_url` into `<img src>` . Verified: thumbnails now render. NOTE: the write-path that double-encodes the value (designer → order item) should also be fixed so new data is clean - the read-side fix is defensive and covers existing + future bad rows.
-  **`fetchChatList` crashed on a null socket** - after the connect-fix above, the chat launcher still threw `Cannot read properties of null (reading 'emit')` from its `mounted` hook on every admin page when the chat server is offline, polluting the global Vue error handler. Added a guard in `chat.ts#fetchChatList` to no-op when the socket isn't connected. Verified: the page-wide Vue error is gone.
-  **`/transactions` rendered a blank page** - the route was a leftover stub (`<template>Transactions</template>`), not linked in the nav, with no loading/empty/error state (violates “no half-built UI”). Since payment transactions live on the Accounting page, the orphan route now redirects to `/accounting` .

Findings (not yet fixed - minor / data, low priority)

-  Dashboard **Quotes** widget shows “ Expired” with an hourglass emoji while every other status (and the Invoices widget) uses a colored dot only - inconsistent label decoration. The emoji comes from backend status label data, not an obvious frontend constant.
-  **Web Leads:** the one seeded lead’s Message column reads “Phone is required.” - looks like a form-validation string was saved as the message (likely stale test data; worth checking the public inquiry form doesn’t persist validation text as the message body).
-  **AI Usage:** “Total Requests 90” but Successful 20 / No-Match 0 / Errors 0 - ~70 requests fall into none of the displayed buckets (status classification gap in the summary cards).

End-to-end FLOWS exercised (data entered + submitted) - all , 0 backend errors

- **Create customer** → “QA Test Co” saved, redirected to detail page.
- **Add newsletter subscriber** → `qa.newsletter@example.com` saved; **audit.log** entry written (validates the audit-channel fix through the real controller, who/what/when captured).
- **Create category** → “QA Test Category” saved, appears active in list (7 → 8).
- **Create quote + add line item** → QUOTE-#0005 created and correctly linked to “QA Test Co” (the customer created above); A4 Poster line item auto-priced at \$69.00, saved.

- **Order status change** → ORDER-#0035 Draft → Open; persisted, logged in the order Activity timeline; customer-email + team notifications toggled off so no external mail was sent.
- **Create email campaign** → “QA Flow Campaign 2” through the 3-step wizard (Audience → Content), saved as Draft (not sent); **audit.log** entry written.

Confirmed: customer → quote linkage and product → line-item auto-pricing work; status transitions persist and are logged; audit logging works end-to-end. `grep -c ERROR|EMERGENCY laravel.log` = 0 across the entire session.

Storefront complex flow + pricing methods (:3001) - 2026-06-06

Pricing methods - all verified on real PDPs

- **fixed** (Premium Business Cards) - ☒ base \$35 + options → **\$50.00**, breakdown correct.
- **area_based** (Custom Flyers) - ☒ computes **\$100.00**, Add to Cart enabled.
- **quantity_based** (Professional Letterhead) - ☒ \$77 @ qty1 → **\$385 @ qty5** (“\$77.00 each” shown); scales.
- **percentage** (Vinyl Banners) - ☒ computes **\$28.15**.
- **formula** (Custom Booklets) - ☒ correctly shows **\$0.00** + “We couldn’t calculate a price... request a quote”, Add to Cart **disabled**, Request-a-Quote offered. (Deeper fix - PDP collecting the formula’s page-count input - still open; the guard is correct UX.)

Full purchase flow: storefront → admin order - COMPLETED

Guest checkout (Premium Business Cards \$50) → placed **ORD-#0036** via Cheque → **appears in admin** with correct customer (QA Storefront Buyer), line item, options (Glossy / 20lb Bond), and gateway.

Confirmed bug - shipping cost dropped on order creation (revenue loss)

Storefront cart/checkout charged **\$100 shipping (total \$150)** and the success page said \$150, but the **admin order Grand Total is \$50** and its Shipping section shows **Cost \$0.00 / Method “-”**.

- Root cause: `app/Services/Storefront/Checkout/CheckoutPricingService.php` has **no shipping handling at all** - order totals are built without shipping. The cart shows shipping via a *different* service (`StorefrontCartService`). The checkout payload sends only the `shipping_method` key (`checkout.vue:1468`); the order-pricing service never resolves it to a cost.
- Impact: customers are shown/agree to a total that the placed order doesn’t reflect; shop is undercharged.
- Fix needs backend work in CheckoutPricingService to resolve the selected shipping method → cost and add it to the order total + persist `shipping_cost` . **Requires a test** (pricing/checkout rule) - left for a focused PR rather than a blind end-of-session patch. (Matches the known cart ↔ checkout mismatch.)

☒ **FIXED - order total now includes shipping + subtracts coupon discount**

The bug below is now fixed (pre-launch system, so correcting the canonical total is appropriate):

- `Order::getTotalAmountAttribute()` now returns `lineItems+itemTax + shipping_cost - Σcoupon.discount_amount` (rounded). Uses the loaded `coupons` relation when present (eager-loaded by the order list/detail queries) to avoid N+1.
- `CheckoutPricingService::resolveOrderGrandTotal()` now delegates to `Order::total_amount` (single source of truth).
- `StorefrontCheckoutController` now **persists** `shipping_cost` on the order, resolving the selected `flat_rate_{uuid}` method via `ShippingRate::effectiveCharge(subtotal)` (same calc the cart uses); carrier/other methods resolve to 0 (documented limitation).
- Tests: `tests/Unit/Orders/OrderTotalTest.php` (4 cases: items+tax, +shipping, +coupon, unit-price fallback) - all green. Re-ran orders/pricing/checkout suites (58 tests) - all pass. Impact assessment: **0 existing orders had shipping_cost set** and only **2 dev orders have coupons** (their totals are now correctly reduced). Effect: new storefront orders charge the

shipping the customer saw, and redeemed coupons actually reduce the total (payment amount, B2B credit and invoices all read this accessor, so they're now consistent).

- NOTE: a pre-existing, unrelated test error exists in

`tests/Feature/Accounting/InvoicePaymentsInAccountingTest.php:119` (inserts a Payment with a numeric `paymentable_id` into a uuid column) - fails on clean `main` too; not caused by this change.

(original finding, for context) order total ignored shipping AND coupon discount

Root-caused the shipping drop above to a single structural gap that ALSO affects coupons:

- `Order::getTotalAmountAttribute()` (app/Models/Order.php:264) and `CheckoutPricingService::resolveOrderGrandTotal()` both compute the order total as `sum(line_amount + item.tax)` **only** - no shipping, no discount.
- `StorefrontCheckoutController Order::create` (line ~453) never persists a `shipping_cost`, and although a coupon is *recorded* via `couponService->redeem()` (line ~504), the discount is **never subtracted** from the order total.
- Net effect for storefront orders: **shipping is dropped (undercharge)** and a **redeemed coupon shows the full, undiscounted total** (customer expected a discount; the order/admin shows full price). Same “recorded-but-not-applied” class as the known markup-dropped bug.
- Fix = a focused backend PR (persist `shipping_cost` on order; make the total accessor add shipping and subtract coupon discount) **with tests** - high blast radius (every order/invoice/accounting view), so deliberately not patched blind here. Coupon E2E was verified at the code level after the live storefront session got flaky (stray designer/variant tabs); the structural gap is unambiguous.

✓ Fixed this session - \$0/price-unavailable item could be checked out

The PDP blocks adding a \$0 formula product, but a stale/zero-priced line already in the cart flowed all the way into checkout (cart had a \$0 Custom Booklets → Subtotal \$0 + \$100 shipping = checkout-able). Extended `checkout.vue#blockingCartIssues` to flag any line where both `total_price` and `unit_price` are ≤ 0 , mirroring the PDP guard. **Verified:** checkout now shows “We couldn’t calculate a price for Custom Booklets. Please request a quote for it or remove it to continue.” with Place Order disabled; removing the item re-enables checkout (no over-blocking).

⚠ Other storefront findings (not fixed)

- **Default payment method = “Authorize.Net” which shows “inline checkout is not yet available.”** A non-working gateway is the default; should default to an enabled one (Cheque/Stripe). It does guide the user to pick another, so not silently broken - but a poor default.
- **Offline (Cheque) order success page over-claims “Payment Successful / Paid”** while the admin correctly shows the payment as “**Initiated**” (cheque not actually received). The customer-facing success copy should say “Order placed - payment pending” for offline methods.
- **Cart remove** updates the count/subtotal immediately but the removed line card lingers until reload (minor reactive-list refresh lag).

Pages reached & verified this session (all ✓ unless noted)

Dashboard · Action Center · Customers (list/new/view) · Quotes · Invoices · Orders (list/detail) · Products · Categories · Master Options · Size Guides · Imposition (Job Watch) · Jobs (board) · Proofs · Email Campaigns · Newsletter Subscribers · Marketing Segments · Web Leads · AI Usage · Store Management (+ store overview, Pages & CMS, page-builder editor) · Accounting · Business Accounts · Settings (Account, Payment Gateway, Team/Users/Roles) · Transactions (now redirects). Skipped per request: Chat module.

Dashboard & top-level

- `/` `Dashboard`
- `/action-center/`
- `/chat/`
- `/tasks/` `/tasks/[id]`
- `/transactions/`
- `/web-leads/` `/web-leads/[id]`

Customers

- `/customers/` `/customers/new/` `/customers/view/[id]` `/customers/import`
- `/business-accounts/` `/business-accounts/approvals` `/business-accounts/[id]`

Sales: Quotes / Invoices / Orders / Proofs / Jobs

- `/quotes/` `/quotes/[id]` `/quote`
- `/invoices/` `/invoices/[id]`
- `/orders/` `/orders/[id]`
- `/proofs/` `/proofs/[id]`
- `/jobs/` `/jobs/new` `/jobs/[id]`

Catalog: Products / Categories / Options / Sizes

- `/products/` `/products/create` `/products/[uuid]/edit`
- `/products/[uuid]/options/` `/products/[uuid]/pricing-rules/` `/products/[uuid]/sizes/`
- `/categories/` `/categories/create` `/categories/[uuid]`
- `/subcategory/` `/subcategory/create`
- `/master-options/` `/master-options/create`
- `/size-guides/` `/size-guides/create`

Production / Imposition

- `/imposition/` `/imposition/presses/` `/imposition/press-sheets/`
- `/imposition/templates/` `/imposition/gang/new`

Marketing

- `/email-campaigns/` `/email-campaigns/new` `/email-campaigns/[id]`
- `/campaign/` `/campaign/new`
- `/marketing/segments/`
- `/newsletter-subscribers/`

AI

- `/ai/product-builder` `/ai/usage` `/ai/preflight`

Store management

-  `/store-management/stores/`  `/store-management/stores/create`  `/store-management/stores/[uuid]/`
-  `stores/[uuid]/` sub-pages: banners, blog, coupons, faqs, footer, forms, menus, pages, promo-bars, reviews, seo, shipping, testimonials, theme, settings, analytics, regional, designer, newsletter

Accounting

-  `/accounting/`
-  `/secret/.../expense/`

Settings

-  `/setting/` + account, ai-usage, automation, billing, categories, console, designer, emails, inquiry, integrations, log, notifications, payment-gateway, security, Team (Users/Roles), templates, third-party-services, workflows

Account

-  `/account/general`  `/account/devices`

Feedback Capture Readiness

Source: readme/research/qa-launch/qa_section_feedback.md

IN SHORT — KEY TAKEAWAYS

This document checks whether the platform is ready to hear from customers — and the answer is: **not yet**. Right now the shop can take orders and payments, but it has no way to ask buyers “were you happy?”, no way to catch when a checkout breaks, and no way to tell the store owner when something goes wrong. **If problems appear, they will pile up silently until customers stop coming back — which is the worst way to find out.**

The main parts explained simply:

- **Post-delivery satisfaction survey (blocker)** — After a customer gets their printed order, there is no automatic “How did we do?” email. The founder has no tool to find out if the product is actually working for real buyers. This is the single most important thing to fix before launch.
- **Checkout and payment error tracking** — When a customer’s payment fails or checkout breaks, nothing records it. So the store owner sees “people abandoned their cart” but cannot tell if they left on purpose or because something broke. The fix is to track those errors separately so the real cause is visible.
- **Abandoned cart recovery** — When a customer fills a cart but never buys, there is no follow-up email to bring them back. For a print store where customers put real design effort into their cart, this is direct lost revenue.
- **Owner health dashboard** — The analytics page shows sales numbers but cannot answer “Why did people not finish checkout?” or “Are customers complaining about print quality in reviews?” A new “Friction and Health” section is needed so the owner can spot problems without digging through logs.
- **Review and message notifications** — Customers can write product reviews and send order messages, but the store owner is never notified. A complaint about a bad print job, or a one-star review, can sit unread for weeks.
- **Error alerts** — When something breaks on the site (server errors, PDF failures), the owner only sees a wall of technical code they cannot read. No plain-English alert is sent, and errors are never grouped or summarized.

What to do with this: Fix the post-delivery “How did we do?” email first — without it, launch is flying blind. Then make sure the owner gets notified when a customer leaves a review or sends a message. Everything else can follow after those two are live.

Assessment date: 2026-06-15 **Audience:** Founder + engineering **Scope:** How well the platform captures customer satisfaction signals, friction signals, and operational error signals — given that the primary users are **non-technical store owners who will not file good bug reports** and have **never used the product in production**.

Verdict

NOT READY — the platform is effectively blind to customer satisfaction and on-platform friction.

Today the system can *transact* (orders, payments, shipping notifications all exist) but it cannot *listen*. There is no mechanism to ask a customer “how did we do?”, no signal when a checkout fails, no recovery for abandoned carts, and no plain-language health surface for the owner. The one inbound channel that does exist for customers — order messages and product reviews — has **no admin-side delivery**: a customer can write “my order has color banding” or leave a 1-star review, and the store owner is never notified. Combined with an error console that dumps raw `SQLSTATE` stack traces at a non-technical SMB owner, the net effect is that **problems will accumulate silently until they show up as churn or chargebacks**, which is the worst possible failure mode for a first-cohort launch.

The single must-fix before onboarding real stores is the **post-delivery satisfaction loop** (blocker), because without it the founder has no instrument to learn whether the product is actually working for end customers. The cluster of “owner is never

notified” gaps (pending reviews, inbound order messages, error spikes) is the next tier — each one converts a recoverable issue into a silent abandonment.

One stale-doc correction up front: earlier notes implied no shipping/payment-failure email scaffolding exists. **Re-verified** — **EmailTemplateEnum.php** **does ship** **ORDER_SHIPPED (line 29)** and **PAYMENT_FAILED (line 33)**. Those are transactional, not feedback-capture, templates — but the email-template plumbing they ride on is the recommended insertion point for the missing survey/recovery templates, so the lift is smaller than previously assumed.

Prioritized findings

Severity	Finding	Where	Type	Confidence	Recommendation
Blocker	No post-delivery satisfaction survey or NPS mechanism	ABSENT (no model/job/template); <code>EmailTemplateEnum.php:8-37</code>	feedback-gap	confirmed-in-code	Add post-delivery survey job + email template + <code>OrderFeedback</code> model (rating, reason_category, raw text)
High	Checkout & payment errors not tracked as friction	<code>useGtm.ts:274-310</code> ; <code>StorefrontCheckoutController.php:128-140</code>	silent-lie	confirmed-in-code	Add <code>trackCheckoutError</code> / <code>trackPaymentFailure</code> GTM helpers; fire on validation, gateway, inventory failures
High	Abandoned-cart recovery flow missing entirely	ABSENT (no model/job/template); <code>EmailTemplateEnum.php:8-37</code>	feedback-gap	confirmed-in-code	Track cart-abandoned timestamp, daily <code>AbandonedCartRecoveryJob</code> , <code>cart_recovery</code> template + GTM event
High	No friction/health analytics for owners	<code>analytics/index.vue:1-236</code>	feedback-gap	confirmed-in-code	Add “Friction & Health” card: checkout error rate, top error types (plain English), cart-abandon rate, review trend, zero-result searches
Medium	Review backlog invisible; no pending-review notification	<code>StorefrontProductReviewController.php:79-105</code> ; <code>reviews/index.vue</code>	support-gap	confirmed-in-code	<code>ReviewSubmittedNotification</code> to owner; nav badge; per-store auto-approve; 60-day auto-publish
Medium	No error-monitoring alerts; errors require manual log grep	<code>ErrorLogService.php:47-69</code> ; <code>setting/log/index.vue:37-118</code>	support-gap	confirmed-in-code	Email owner on >5 identical errors/hr; 7-day trend; plain-English summaries; conditional Sentry
Medium	Order messages wired in storefront but no admin reply panel	<code>OrderMessageThread.vue:1-320</code> ; <code>profile/orders/[id].vue:204-206</code> ; <code>OrderMessage.php:33</code>	missing-loading-empty	likely	Build admin order-message panel with reply, new-message notification, resolve/close, audit trail
Medium	Validation failures silently dropped, not logged	<code>CheckoutRequest.php:25-60</code> ; <code>ErrorLogService.php:35</code>	silent-lie	confirmed-in-code	Log validation failures as <code>source=validation</code> ; GTM <code>checkout_validation_error</code> ; surface most-failed field
Low	No review sentiment/category tagging	<code>ProductReview.php:18-35</code> ; migration <code>:22-25</code>	feedback-gap	confirmed-in-code	Auto-tag on create (keyword → category); <code>review_categories</code> JSON; breakdown card
Low	No helpdesk/support-ticket system	ABSENT (no Ticket/SupportCase model)	support-gap	confirmed-in-code	Add <code>SupportTicket</code> model (status/priority/category/SLA timestamps) + storefront “Contact Support”
Low	No auto-flagging of at-risk orders	ABSENT (no <code>Order.risk_level</code>)	feedback-gap	confirmed-in-code	<code>Order.risk_flags</code> JSON + <code>RiskScoringService</code> + “At-Risk Orders” list + daily digest
Low	No heatmap / session-replay integration	ABSENT (no Hotjar/Clarity/LogRocket)	feedback-gap	confirmed-in-code	Optional: replay-service ID in <code>StoreAnalyticsSetting</code> , conditional SDK load
Low	Raw technical error output shown to owners	<code>setting/log/index.vue:71,93-97</code>	bad-error-state	confirmed-in-code	<code>ErrorSummaryService</code> maps exceptions → plain English; full trace in collapsed “Debug Info”

Severity	Finding	Where	Type	Confidence	Recommendation
Low	No GTM events on checkout/payment errors (funnel)	<code>useGtm.ts:274-310</code> ; <code>checkout.vue</code>	feedback-gap	confirmed-in-code	Add <code>checkout_abandoned</code> / <code>checkout_error</code> / <code>payment_error</code> with abandonment-point field

Note: the “high” finding on checkout-error tracking and the “low” finding on missing GTM funnel events are two views of the same root cause (`useGtm.ts` has no error/abandon helpers). They are listed separately because the high item is about *missing the friction signal entirely* and the low item is about *funnel-step attribution*; fixing the high item should be scoped to deliver both.

Detail on the most important items

1. Blocker — there is no way to ask a customer if they were happy

This is the defining gap for a pre-launch readiness review. `EmailTemplateEnum.php` enumerates 15 templates (lines 8–37) and **none** is a satisfaction, NPS, or post-delivery feedback email. There is no `PostDeliverySurvey` / `CustomerSatisfactionSurvey` model, no job in `app/Jobs/` to send a feedback request, and no `OrderDelivered` listener to trigger one. The order lifecycle runs to completion — the `ORDER_SHIPPED` template (line 29) even fires — and then the conversation simply ends.

For a first cohort this is not a “nice to have”: it is the founder’s only instrument for learning whether the product works for the people who actually receive the printed goods. Store owners are non-technical and will not build their own survey tooling.

Recommendation: add an `OrderFeedback` model (`rating` , `reason_category` ∈ {quality, speed, price, service}, free-text), an email template, and a job dispatched 3–5 days after `ORDER_SHIPPED` . Make it a 1-click NPS (0–10) with an optional reason field. Because the email-template and shipped-event plumbing already exist, this rides existing rails rather than introducing a new subsystem.

2. High — checkout and payment failures vanish; they look like cart abandonment

`useGtm.ts` instruments the *happy path* — `trackBeginCheckout` (line 274), `trackAddShippingInfo` (287), `trackAddPaymentInfo` (301), `trackPurchase` (315) — but has **no** `trackCheckoutError` or `trackPaymentFailure` helper. On the backend, `StorefrontCheckoutController.php` catches `PaymentSessionException` and returns a 422 (line 139) without emitting any analytics event or metric. And `ErrorLogService::shouldPersist()` explicitly excludes `ValidationException` (line 35), so a customer who fails checkout validation leaves **no trace anywhere** — not in GTM, not in `error_logs` .

The consequence is a *silent lie* in the owner’s data: a customer who enters a bad card at the payment step, sees a toast, and leaves is indistinguishable from a customer who casually abandoned a cart. The owner sees “abandonment” and may “fix” it with discounts when the real problem is a broken payment gateway.

Recommendation: add `trackCheckoutError(reason, stepName)` / `trackPaymentFailure` firing a `checkout_error` event with `step` (shipping/payment/validation), `reason_code` , and `error_message` ; emit on validation failure, gateway timeout, and inventory failure. Track these distinctly from generic 500s so conversion blockers are isolable. This is the same work that closes the low-severity “no funnel-drop events” finding.

3. High — abandoned carts are never recovered

No `AbandonedCart` / `CartAbandonment` model exists; none of the 18+ jobs in `app/Jobs/` reference cart recovery; and `EmailTemplateEnum.php` (re-verified lines 8–37) has no `cart_recovery` template. A cart can sit in the database indefinitely

with no order and no follow-up. For a print-on-demand store where carts carry custom-designed, high-intent items, this is direct revenue left on the floor and — relevant to this section — a lost signal about *where* purchase intent breaks down.

Recommendation: stamp a `cart_abandoned` timestamp when a cart is >24h old with no order; schedule a daily `AbandonedCartRecoveryJob`; add a `cart_recovery` template (optionally with a discount code); and fire a `cart_abandoned` GTM event so the abandonment point is visible alongside finding #2.

4. High — the owner’s analytics page can’t answer a single friction question

`analytics/index.vue` (lines 1–236) renders revenue, order count, customer count, top products, order-status breakdown, and recent orders — a *vanity* dashboard. It contains no funnel, no error metrics, no review trend, and no zero-result-search view. The only place errors are visible is `/setting/log`, which (see #7) is raw technical output a non-technical owner cannot read.

So the owner can see *that* money came in, but not *why more didn’t*: “What % of checkouts fail at payment?” and “Are print-quality complaints showing up in reviews?” are both unanswerable.

Recommendation: add a “Friction & Health” card surfacing last-7-day checkout error count and %, top 5 error categories in plain English (payment / validation / server), cart-abandonment rate, average review rating trend, and zero-result search terms. This card is the consumer for the events added in #2 and #3 and the summaries in #6/#7 — sequence it after those.

5–6. Medium — three “the owner is never told” gaps compound

Three findings share one failure pattern: an inbound signal exists but **never reaches the owner**.

- **Pending reviews** (`StorefrontProductReviewController.php:79-105`): a submitted review is created `status='pending'` and the customer is told “pending moderation” (line 97), but no `ReviewSubmittedNotification` fires. Reviews rot in limbo until the owner happens to visit `/reviews`. Add owner notification, a nav badge, a per-store auto-approve for verified purchases, and a 60-day auto-publish backstop.
- **Inbound order messages** (`OrderMessageThread.vue:1-320`, rendered at `profile/orders/[id].vue:205`): the *storefront* has a full send/receive/attachment thread, and `OrderMessage.php` carries a `user_id` (line 33) clearly intended for staff replies — but there is **no admin-side panel or controller** to read or answer them (confidence: likely; storefront wiring confirmed, admin absence inferred from missing route/controller). A customer reporting “color banding” gets silence, then leaves. Build the admin reply panel with new-message notification, resolve/close, and audit trail.
- **Error spikes** (`ErrorLogService.php:47-69`, `setting/log/index.vue:37-118`): all ≥500 errors persist, but there is no alerting or aggregation. Fifty identical “PDF generation timeout” errors over a week produce zero owner awareness. Add dedup-based alerting (>5 identical errors/hr → email), a 7-day trend, plain-English summaries, and conditional Sentry auto-enable. Note `pdf-service/src/lib/sentry.js` exists but is opt-in (off unless `SENTRY_DSN` is set).

7. Low but launch-relevant — the error console speaks developer to shopkeepers

`setting/log/index.vue` renders `exception_class` (line 71), `file:line` (line 93), and full stack traces (line 97) verbatim — e.g. `SQLSTATE[HY000]: General error: 2006 MySQL server has gone away`. This violates the project’s own §0 “never surface raw technical output” rule directly in the owner-facing admin. A non-technical owner cannot tell a transient DB blip from a code defect, so they take no useful action.

Recommendation: an `ErrorSummaryService` that maps exception types to plain language (`SQLSTATE[HY000]` → “Database connection issue”; `PaymentGatewayException` → “Payment provider unreachable”) with the full trace tucked into a collapsed “Debug Info” section for support. (Minor note: the example error string references MySQL, but this platform runs PostgreSQL per `CLAUDE.md §6`; the mapping table should be written against Postgres `SQLSTATE` codes.)

Readiness recommendations (added — not findings)

These are obvious sequencing/launch-hygiene calls, labelled as recommendations rather than discovered defects:

1. **Minimum bar for first-cohort launch:** ship #1 (post-delivery survey) and the owner-notification half of #5/#6 (pending reviews + inbound order messages + error-spike email). Without these three, the founder is launching blind and customers' complaints land in a void.
2. **Build the analytics “Friction & Health” card last in this workstream (#4)** — it is the display surface that consumes the events and summaries created by #2, #3, #6, #7. Building it first yields an empty card.
3. **Treat the GTM helper as one change**, delivering both the high-severity friction signal (#2) and the low-severity funnel attribution (#13) from a single `useGtm` edit.
4. **Defer #9 (helpdesk), #10 (risk scoring), #11 (session replay), and #8 (review sentiment) to post-launch.** They are genuine gaps but not blockers for a first cohort, and several (replay, sentiment) are better validated against real traffic than built speculatively.
5. **Whatever feedback storage is added** (`OrderFeedback` , `cart_recovery` , validation logging) must follow the project's silent-drop invariant: every captured field must validate → save → read back → render, with a test that asserts persistence. The validation-drop finding (#7/#8 in the table) is itself an instance of this class.

Critical money-path QA (signup → product config → cart → checkout → payment → order → fulfillment)

Source: [readme/research/qa-launch/qa_section_money_paths.md](#)

IN SHORT — KEY TAKEAWAYS

This document checks whether your store can actually take money and turn it into a finished order — from signup, through adding a product to cart, paying, and getting it printed and shipped. The result is: the store is **NOT** ready to launch yet. There are four serious problems that could hurt real customers or lose real money on day one.

The main parts explained simply:

- **The four must-fix problems (blockers)** — Four issues are so serious that any one of them alone would stop a launch. They are: (1) stock is never reduced when someone buys, so two customers can buy the same last item; (2) the “forgot password” link lies — it shows a success message but never sends an email, so any customer who forgets their password is permanently locked out; (3) there is a hidden backdoor code (`702702`) baked into the system that lets anyone bypass admin security; (4) when a product requires a customer to upload their artwork, that requirement is silently ignored — orders reach production with no artwork attached.
- **Signup and login** — Beyond the two blockers above, the OTP (one-time code) resend has no limit, so someone could flood an inbox. Admin password rules are also weaker than they should be.
- **Product page and pricing** — The pricing math is mostly correct, but extra charges based on the size a customer picks (for example “+\$5 for A4”) never actually apply — every customer pays the base price regardless of size chosen.
- **Cart** — Items stay “reserved” in the cart but stock is never actually held. If someone in another browser tab buys the last item, the cart doesn’t know. Also, if a customer shops at two different stores on the same device, their carts can mix together.
- **Checkout and addresses** — Address fields (shipping, billing) are not properly checked by the server. A customer can submit blank or broken address data and the order saves it anyway. Shipping costs can silently change to \$0 between the cart and the checkout summary.
- **Payments** — When a customer pays by cheque (offline), the screen wrongly shows “Payment Successful” even though no money has arrived yet. Orders paid by cheque then sit stuck forever with no reminder to the store owner. A missing configuration value (webhook secret) can cause all online card payments to get stuck in “pending” too.
- **Order confirmation email** — The email is sent before payment is confirmed, and it shows the wrong total (it leaves out shipping, tax, and any discounts).
- **Shipping and fulfillment** — This is the most incomplete part. There are no order statuses for “In Production,” “Shipped,” or “Fulfilled.” The shipping carrier connection (EasyPost) is built but not turned on, so all orders ship at a flat rate only. Tracking numbers must be typed in by hand. Files needed to print an order can be missing with no warning.

What to do with this:

Fix the four blockers first — stock deduction, real password reset, remove the backdoor code, and stop dropping required artwork uploads. After those are resolved, the next most urgent step is fixing the order confirmation email (send it only after payment, and include the correct total). The fulfillment/production stage needs a dedicated build sprint before the store can handle real print orders end to end.

Launch-readiness assessment — Print-Flow-360 — 2026-06-15 Source probes: [/tmp/qa_money.json](#) (59 findings, 7 stages) + [/tmp/qa_state.json](#) (14 cross-cutting loading/empty/error + mobile findings). Deduped where two probes hit the same issue. Severity legend: ● blocker · ● high · ● medium · ● low

Verdict

The money path is NOT launch-ready. Four blockers sit directly on the spine that takes money and turns it into a fulfilled order, and any one of them is a launch-stopper on its own:

1. ● **Inventory is never decremented** (`StorefrontCheckoutController.php` checkout pipeline; `UpdatePaymentAndOrder.php:155-166`). `ProductInventoryService::deductStock()` exists but is never called from checkout or payment-capture. Two customers can buy the same last unit; the platform will cheerfully oversell every limited-stock product on day one.
2. ● **Customer password reset is a complete fake** (`CustomerAuthController.php:497`). The endpoint returns a green “we sent you a link” success but only logs a `TODO` . No token, no email. Any customer who forgets their password is permanently locked out — a guaranteed support flood at launch.
3. ● **A hardcoded master-OTP backdoor** (`AuthController.php:368`) — `env('MASTER_OTP', 702702)` lets anyone who knows the constant `702702` clear the 2FA check on **any** admin account. This is a credential-bypass backdoor shipping to production.
4. ● **Required file-upload product options are silently dropped** (`productInfo.ts:139-146, 709-714`). When `allow_custom_file_upload=false` , a required file option is filtered out *before* the required check, so Add-to-Cart succeeds with the required artwork never collected — the order reaches production unfulfillable.

Beyond the blockers, the **order-to-production spine is structurally incomplete**: no PRODUCTION/SHIPPED statuses, EasyPost built but unwired (flat-rate only), tracking numbers hand-typed, no partial fulfillment, and print-ready PDFs that can be NULL at checkout. Layered on top is a pervasive **silent-lie / fake-success class**: offline (cheque) orders show “Payment Successful” while `payment_status='pending'` , the confirmation email fires at draft time (“being processed”) before any payment and shows the wrong total, and several checkout address/payment fields skip server validation. The two strongest stages are the **price-calculator math** (largely correct, with two real bugs) and the **cart ↔ checkout tax formula** (verified to match) — but those are the only stages that read as close to shippable.

Prioritized findings table

All findings from both probes, deduped (the offline “Payment Successful” lie and the shipping-\$0 drop were each found by two probes — noted 🗖 and confidence raised).

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
●	Stock inventory never decremented at order/payment	7 Fulfillment	<code>StorefrontCheckoutController.php:54-612 ; UpdatePaymentAndOrder.php:155-166</code>	money-mismatch	confirmed	Call <code>deductStock()</code> in payment-captured flow, wrapped in a transaction; block oversell
●	Customer password reset is a TODO that lies “link sent”	1 Signup/Auth	<code>CustomerAuthController.php:497</code>	silent-lie	confirmed	Implement token+email+reset endpoint, or hide the feature until built
●	Hardcoded master-OTP <code>702702</code> bypasses admin 2FA	1 Signup/Auth	<code>AuthController.php:368</code>	silent-lie	confirmed	Remove backdoor; if needed, gate to system-admins with rate-limit + audit
●	Required file-upload option silently dropped when uploads disabled	2 PDP/Pricing	<code>productInfo.ts:139-146, 709-714</code>	silent-lie	confirmed	Keep file options in validation; show “file required but unavailable” error
●	Per-size price markup never applied (size_id never sent)	2 PDP/Pricing	<code>productInfo.ts:940-946 ; ProductPricingCalculator.php:408</code>	money-mismatch	confirmed	Send <code>selected_size_id</code> in calc + configure context
●	Print-ready PDF can be NULL at checkout, blocks production download	7 Fulfillment	<code>StorefrontCheckoutController.php:659-668</code>	silent-lie	confirmed	Auto-generate PDF at checkout OR reject order if missing
●	Preflight validation disabled, never auto-runs (72 DPI/RGB to production)	7 Fulfillment	<code>PreflightController.php:38-40</code>	bad-error-state	confirmed	Enable flag; gate checkout on ≥ 300 DPI with plain-language error
●	EasyPost carrier rates built but never wired into checkout	7 Fulfillment	<code>EasyPostShipping.php:40-104 ; StorefrontCheckoutController.php</code>	missing-loading-empty	confirmed	Add shipping-rates endpoint; resolve cost from carrier, not flat-rate table

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
			p:486-496			
●	No PRODUCTION/FULFILLED/SHIPPED order statuses	7 Fulfillment	OrderEnum.php:5-14	onboarding-gap	confirmed	Seed core lifecycle statuses; sync with PrintJob transitions
●	Tracking number hand-typed; no label generation / carrier API	7 Fulfillment	OrderController.php updateShipping()	missing-loading-empty	confirmed	Create Shipment on READY_TO_SHIP via EasyPost; auto-fill tracking + poll
●	No partial fulfillment / split-shipment; orders atomic	7 Fulfillment	Order.php ; Item.php:1-76	field-drop	confirmed	Add Shipment model + per-item fulfilled_quantity (post-launch feature)
●	Confirmation email shows item subtotal only (drops shipping/tax/coupon)	6 Order	StorefrontCheckoutController.php:580	money-mismatch	confirmed	Use \$order->total_amount after save, not \$items->sum('amount')
●	Confirmation email "received & being processed" sent at draft, before payment	6 Order	StorefrontCheckoutController.php:571-610 (send @596)	silent-lie	confirmed	Send on payment_status='paid', or reword to "awaiting payment"
●	Offline (cheque) order shows "Payment Successful" but status=pending 🚩(2 probes)	5 Payment	order-success.vue:779-798, 638-655	silent-lie	confirmed	Relabel: "Order Submitted / Awaiting Cheque Payment" + mailing instructions
●	Shipping silently \$0: nullable shipping_method + carrier keys never priced 🚩(2 probes)	4 Checkout	CheckoutRequest.php:40-42 ; StorefrontCheckoutController.php:488-496 ; checkout.vue:1486-1488	money-mismatch	confirmed	Require method when cart showed shipping; price non-flat-rate methods
●	Missing webhook_secret hangs order in 'pending' forever	5 Payment	RazorpayGateway.php:348-350 ;	silent-lie	confirmed	Validate/warn on missing webhook_secret at gateway config save time

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
			PaymentWebhookController.php:23-46			
●	Cheque orders stuck 'pending' forever, no automation/reminder	5 Payment	ChequeGateway.php:117-120 ; UpdatePaymentAndOrder.php	silent-lie	confirmed	Action Center card for pending cheques + daily admin digest
●	new_shipping_address / new_billing_address arrays have NO FormRequest rules	4 Checkout	CheckoutRequest.php:25-59	silent-lie	confirmed	Add required_with rules for address_line1/city/zip/country
●	Missing address fields default to empty string, save invalid address	4 Checkout	StorefrontCheckoutController.php:427-432	silent-lie	confirmed	Drop null-coalescing; validate before Address::create()
●	Quote endpoint shares the same missing address validation	4 Checkout	StorefrontCheckoutController.php:687-691, 720-757	silent-lie	confirmed	Apply shared address rule set to quote path
●	CheckoutCustomFieldsTest fails — draft order returns 400	4 Checkout	tests/Feature/Storefront/CheckoutCustomFieldsTest.php:113	bad-error-state	confirmed	Diagnose 400 (cart seeding / store-header mismatch); make test green
●	Cart vs checkout tax double-count risk when prices_include_tax	3 Cart / 4 Checkout	StorefrontCartService.php:54-61 ; checkout.vue:789	money-mismatch	likely	Verify resolvedTotal doesn't add cartTax when subtotal already includes it
●	Cart shows first shipping method, checkout lets user pick a different one	3 Cart	StorefrontCartService.php:102-	money-mismatch	confirmed	Pre-select cart's method; show delta when changed

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
			104 ; checkou t.vue:77 5-790			
●	Guest cart localStorage key not tenant-scoped (cross-store collision)	3 Cart	cartSto re.ts:4, 48-57	silent-lie	confirmed	Scope key by hostname: storefront-cart- session-\${hostname}
●	Admin OTP resend has no rate limiting (inbox flood / DoS)	1 Signup/Auth	routes/ api.php: 116	other	confirmed	Add throttle:3,1 + per-email/device throttle
●	No Stripe signed-return fallback (Razorpay only) → stuck pending in dev	5 Payment	StripeG ateway.p hp (no verifyR eturnPar ams)	missing-loading-empty	likely	Implement verifyReturnParams() mirroring Razorpay
●	Storefront profile/orders list has no error state (blanks on fetch fail)	6 Order	profile /orders/ index.vu e:357- 371	missing-loading-empty	confirmed	Add error card + Retry, matching admin order/List.vue:108-114
●	Storefront order detail shows “Order not found” on network error	6 Order	profile /orders/ [id].vue :298- 303	missing-loading-empty	confirmed	Track fetch error; show retrievable error card vs “not found”
●	Checkout config load fails silently → broken empty form	4 Checkout	checkou t.vue:11 63-1194	bad-error-state	confirmed	Add checkoutConfigError ; show error card before form sections
●	Quantity min/step auto-clamps with no feedback (1 → 250)	2 PDP/Pricing	Product Quantity Selector .vue:97- 104	other	confirmed	Toast “Quantity adjusted to 250 (sold in multiples of 250)”
●	Custom-size bounds validated client-only, not backend	2 PDP/Pricing	Product SizePick er.vue:1 63-192 ; Product PricingC alculato r.php:37 1-394	field-drop	confirmed	Enforce min/max bounds in prepareContext()
●	No validation that required file options are collectible (admin blind)	2 PDP/Pricing	product Info.ts: 53-66, 139-142	support-gap	confirmed	Customer + admin warning when required file option can't be collected

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
●	\$0-price guard shows generic "request a quote", no diagnosis	2 PDP/Pricing	<code>productInfo.ts:372-387 ; ProductInfoPage.vue:199-203</code>	feedback-gap	confirmed	Improve message + admin diagnostic for misconfigured formula vars
●	Item notes save on blur with no loading state; lost on timeout	3 Cart	<code>cart.vue:146-148 ; useCart.ts:593-611</code>	bad-error-state	confirmed	Add per-item saving state; validate field in response
●	Availability checked at add-time, not re-validated at checkout (oversell)	3 Cart	<code>StorefrontCheckoutController.php:351-369 ; StorefrontCartService.php:145-172</code>	money-mismatch	likely	Re-validate availability in <code>createOrderDraft()</code> ; reject if unavailable
●	CartItem availability check is display-only; no reservation/block	3 Cart	<code>StorefrontCartService.php:145-158</code>	silent-lie	confirmed	Re-validate at order creation or add short-lived reservation
●	Carrier shipping methods record <code>shipping_cost=0</code> on order	4 Checkout	<code>StorefrontCheckoutController.php:486-496</code>	field-drop	confirmed	Compute carrier cost or mark nullable + document
●	Razorpay save accepts empty <code>key_secret/webhook_secret</code> as valid	5 Payment	<code>RazorpayGateway.php:281-284</code>	silent-lie	confirmed	<code>validateConfiguration()</code> should require <code>webhook_secret</code> for webhook gateways
●	<code>markAsPaid()</code> fires triggers inside payment DB transaction	5 Payment	<code>UpdatePaymentAndOrder.php:155-166 ; Order.php:190-202</code>	other	confirmed	Move <code>TriggerDispatcher::dispatch()</code> outside the transaction
●	Order <code>status='paid'</code> decoupled from <code>payment_status</code> (mark paid w/o	5/6	<code>OrderController</code>	silent-lie	confirmed	Warn/confirm on <code>status='paid'</code> when <code>payment_status≠paid</code> ; document

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
	payment)		.php:1467			orthogonality
●	Missing <code>order_uuid</code> in redirect URL breaks polling (stuck pending page)	5 Payment	<code>order-success.vue:1-3</code> ; <code>CheckoutPaymentService.php:89-96</code>	missing-loading-empty	likely	Always append <code>order_uuid</code> ; implement by-number resolver TODO
●	Order-success spinner 5-90s with no progress/ETA feedback	6 Order	<code>order-success.vue:373-425, 643</code>	missing-loading-empty	confirmed	Show elapsed time / progress bar / “taking longer” at 30s
●	Order-success timeout hides error behind computed text, spinner spins on	6 Order	<code>order-success.vue:535-545, 657-692</code>	bad-error-state	confirmed	Distinct timeout state + “Contact Support” button
●	Offline order: cart cleared + UI ‘paid’ while backend stays ‘pending’	5 Payment	<code>order-success.vue:779-798</code> ; <code>StorefrontCheckoutController.php:93-127</code>	silent-lie	confirmed	Mark offline paid at creation OR don’t clear cart/fire GTM until paid
●	Unsafe <code>isOrderOwner()</code> : failed <code>markOwned()</code> → friend’s URL wipes cart	6 Order	<code>order-success.vue:390-416</code> ; <code>checkout.vue:1620</code>	silent-lie	likely	Server-side order-owner check; log <code>markOwned</code> failures
●	Quote submission has no loading state (duplicate-quote risk)	4 Checkout	<code>checkout.vue:1564-1595</code>	missing-loading-empty	likely	Add submitting flag; disable button during call
●	Checkout custom-field errors inline only, no summary banner	4 Checkout	<code>checkout.vue:909-921</code>	feedback-gap	confirmed	Add plain-language summary banner above fields
●	Cart page has no error state for API failures	3 Cart	<code>cart.vue:31-59</code>	missing-loading-empty	likely	Add error card + Retry to cart composable
●	Storefront profile/index stats fail silently (blank tiles)	6 Order	<code>profile/index.v</code>	missing-loading-	confirmed	Check all <code>.ok</code> ; show error card before stats grid

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
			ue:205-219	empty		
●	Admin order list shows “Deleted customer” though snapshot exists	6 Order	nuxt/./order/List.vue:245-254	silent-lie	likely	Expose <code>customer_snapshot</code> in <code>OrderResource</code> as fallback
●	Public order tracking has no rate-limit (enumeration risk)	7 Fulfillment	StorefrontCheckoutController.php:978-1040	support-gap	likely	Add throttle middleware + CAPTCHA on repeated failures
●	Print-ready PDF generation failure is silent (logged, never surfaced)	7 Fulfillment	DesignerController.php:1431-1459	bad-error-state	likely	Add <code>print_ready_status</code> flag; warn designer + checkout
●	B2B domain rejection only at sign-up, not login / admin-created accounts	1 Signup/Auth	CustomerAuthController.php:130-132 vs 262-320	other	confirmed	Enforce domain check in login + Google + admin creation
●	Admin registration uses hardcoded min:8, ignores SecurityPolicyService	1 Signup/Auth	AuthController.php:61	field-drop	confirmed	Use <code>SecurityPolicyService::passwordRules()</code>
●	Mobile: cart/checkout tap targets may be <44px at 375px	3/4	cart.vue quantity + shipping selectors	mobile	needs-runtime	Audit at 375px; enforce min 44x44px
●	“Estimated Total” label misleads (excludes shipping)	2 PDP/Pricing	ProductPriceSummary.vue:8	silent-lie	likely	Relabel “Estimated Subtotal”; note shipping at checkout
●	Missing loading indicator during 300ms price-calc debounce	2 PDP/Pricing	productInfo.ts:836-864	bad-error-state	likely	Set <code>isCalculating=true</code> immediately on change
●	Coupon apply/remove methods exist but no cart UI (dead code)	3 Cart	cartStore.ts:23-24, 459-493 ; cart.vue	field-drop	confirmed	Add coupon input + apply/remove UI to cart summary

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
●	Product placeholder image uses absolute path (breaks in subdir)	3 Cart	<code>cart.vue:89 ;</code> <code>cartStore.ts:161</code>	other	likely	Import asset or use relative path
●	Quantity update success not validated against response field	3 Cart	<code>cart.vue:613-619 ;</code> <code>cartStore.ts:356-358</code>	silent-lie	needs-runtime	Validate returned quantity in <code>applyServerCart()</code>
●	Server allows empty <code>payment_method</code> (client-only guard)	5 Payment	<code>checkout.vue:1314-1321 ;</code> <code>CheckoutRequest</code>	silent-lie	confirmed	<code>payment_method => required string min:1</code> + active-gateway rule
●	Offline payment clears cart before success page loads	5 Payment	<code>checkout.vue:1697-1710 ;</code> <code>order-success.vue:310</code>	missing-loading-empty	likely	Include <code>order_uuid</code> ; defer cart clear until paid; email on create
●	Payment method not validated at draft time (error at intent step)	5 Payment	<code>CheckoutRequest.php:39 ;</code> <code>StorefrontCheckoutController.php:128-140</code>	other	confirmed	Custom <code>ValidPaymentMethod</code> rule scoped to store
●	Redirect-flow cart clear without <code>order_uuid</code> verification (replay)	5 Payment	<code>order-success.vue:800-828</code>	other	likely	Implement by-number resolver with email check
●	Quote custom-field snapshot drops B2B company/department context	4 Checkout	<code>StorefrontCheckoutController.php:673-846 vs 504-505</code>	field-drop	confirmed	Persist <code>company_account_id/department_id</code> on Quote
●	Confirmation email fallback text doesn't state payment-pending	6 Order	<code>StorefrontCheckoutController.php</code>	other	confirmed	Reword fallback or defer email to paid

Sev	Finding	Stage	Where (file:line)	Type	Conf	Recommendation
			p:591-592			
●	OTP logged in plaintext	1 Signup/Auth	AuthController.php:285,346	other	confirmed	Log hash/truncated only
●	Remembered-device OTP = sha256(device_id) only, forgeable if cookie stolen	1 Signup/Auth	AuthController.php:374	other	confirmed	Bind IP/user-agent into hash
●	Design-lock failure at checkout allows instant resubmission spam	4 Checkout	checkout.vue:1480-1484	bad-error-state	likely	Inline error + disable button with tooltip

Per-stage assessment

1. Signup & Auth — ● two blockers, not shippable

Two of the four launch blockers live here. **Password reset is entirely fake** (`CustomerAuthController.php:497`): the endpoint returns a 201 “we’ll email you a reset link” while the body is a `TODO` — no token, no mail. Every customer who forgets their password is locked out with no recovery, and the storefront actively lies to them with a green success message (`login.vue:342`). Separately, a **hardcoded master OTP** (`AuthController.php:368` , default `702702`) defeats admin 2FA for any account — a credential-bypass backdoor that must not ship. The ● unthrottled `resendOtp` (`routes/api.php:116`) is an inbox-flood/DoS vector while its sibling `forgotPassword` is correctly throttled. The remaining items (admin registration ignoring `SecurityPolicyService` , B2B domain enforced only at sign-up, plaintext OTP logging, forgeable device hash) are real but lower-priority hardening. **This stage cannot launch until the password-reset and master-OTP blockers are resolved.**

2. Product configuration / PDP / price calculator — ● one blocker, otherwise the strongest stage

The pricing engine is mostly correct, but one blocker and one high bug break the money math. **Blocker:** required file-upload options are filtered out *before* the required-field check (`productInfo.ts:139-146`), so when `allow_custom_file_upload=false` a product that needs artwork still passes Add-to-Cart with nothing collected — the order is created un-producible and nobody is told. **High:** per-size price markups never apply because the frontend never sends `selected_size_id` , so the backend’s per-size branch (`ProductPricingCalculator.php:408`) is dead — an 8×10 priced “+\$5” is charged at base price end-to-end (PDP, cart, invoice). The rest are UX-grade: silent quantity clamping (1 → 250 with no feedback), client-only size-bounds validation, an ambiguous “Estimated Total” label, and a debounce with no spinner. **Once the file-option and per-size bugs are fixed, this stage is close to shippable** — it has the fewest structural gaps of the seven.

3. Cart — ● several money-mismatch / state gaps, no blocker

No blockers, but three highs cluster on money correctness and tenancy. The **guest cart localStorage key is not tenant-scoped** (`cartStore.ts:4`) — two stores on a shared host collide and overwrite each other’s guest carts. The **cart shows the first shipping method while checkout lets the user pick another** (`StorefrontCartService.php:102-104`), silently changing the total with no delta warning. Availability is checked only at add-time and is **display-only** — never re-validated at checkout (`StorefrontCartService.php:145-158`), so the cart can place an oversold order. Lower-severity: item-notes save on blur with no loading state (lost on timeout), no cart error state on API failure, an absolute placeholder image path, and **dead coupon code** — `applyCoupon` / `removeCoupon` exist in the store but there is no cart UI to invoke them.

4. Checkout (shipping + tax + addresses + custom fields) — ● silent-validation cluster, no blocker but high risk

This is the highest-density high-severity stage after Fulfillment, dominated by **silently-skipped server validation**.

`new_shipping_address` / `new_billing_address` arrays have **no FormRequest rules at all** (`CheckoutRequest.php:25-59`); the controller then null-coalesces missing fields to empty strings (`:427-432`), persisting blank, invalid addresses on the order — the quote path repeats the identical bug (`:687-691`). The **shipping-\$0 drop** (🔍 found by both probes):

`shipping_method` is `nullable` unless required, and only `flat_rate_*` keys are priced (`:488-496`), so a customer who saw \$100 shipping in the cart can place an order with `shipping_cost=0`. A **failing test**

(`CheckoutCustomFieldsTest.php:113` , draft returns 400) signals a live regression in the custom-field checkout path. State gaps: checkout config fails silently to a broken empty form, custom-field errors lack a summary banner, and the tax double-count risk when `prices_include_tax=true` needs a runtime check. **Add a shared address validation rule set and require a shipping method whenever the cart showed one before this stage ships.**

5. Payment & gateways — ● silent-lie cluster around offline/webhook flows

No blocker, but the **truth of payment status is unreliable**. Offline (cheque) orders show “Payment Successful” in the UI (🔍 found by both probes — `order-success.vue:779-798`) while the backend `payment_status` stays `'pending'` with no Payment row — a chargeback/dispute and reconciliation hazard. Cheque orders then sit pending forever with **no automation or admin reminder** (`ChequeGateway.php:117-120`). A **missing webhook_secret** hangs real card orders in `'pending'` indefinitely because the webhook 401s (`RazorpayGateway.php:348-350`), and the gateway-save validation never catches the missing secret. Order `status='paid'` is decoupled from `payment_status` so an admin can mark an order paid with no payment received. Stripe lacks Razorpay’s signed-return fallback (stuck-pending in dev), and `markAsPaid()` fires automation triggers *inside* the payment DB transaction (a trigger failure rolls back the paid status). **The offline-success lie and webhook-secret hang are the items to fix first here.**

6. Order creation & confirmation — ● the confirmation email lies twice

Two highs both sit on the **order confirmation email**. It computes the total as `$items->sum('amount')` (`StorefrontCheckoutController.php:580`), **dropping shipping, tax, and coupons** — the customer is emailed “\$50” while they were charged \$65. Worse, the email fires at **draft creation, before any payment** (`:596`), telling the customer the order is “being processed” even if payment then fails, with no follow-up — a direct support-call generator. The page-state gaps reinforce the “is my order real?” anxiety: the order-success spinner runs 5-90s with no progress indication and hides its timeout behind computed text rather than a “Contact Support” action; the storefront orders list and order-detail pages blank or show “Order not found” on a network error instead of a retrievable error card; and the admin order list shows “Deleted customer” though the snapshot data exists. **Move the email to payment-confirmed and fix the total before launch.**

7. Fulfillment / production / shipping / tracking — ● the most structurally incomplete stage

This is where the platform is least ready and where the remaining blocker lives. **Inventory is never decremented** (`StorefrontCheckoutController.php` + `UpdatePaymentAndOrder.php:155-166`) — `deductStock()` exists but is unreachable from the money path, so the platform oversells by default. Beyond that, the **order-to-production spine is missing its backbone**: there are no PRODUCTION/FULFILLED/SHIPPED statuses (`OrderEnum.php:5-14`), EasyPost carrier rating is fully built but **never wired into checkout** (`EasyPostShipping.php:40-104`) so only flat-rate ships, tracking numbers are hand-typed with no label generation, there’s no partial/split fulfillment, and the print-ready PDF can be **NULL at checkout** (`:659-668`) — staff click “Download Print-Ready” and hit a null path. **Preflight is disabled** (`PreflightController.php:38-40`), so 72-DPI RGB no-bleed artwork reaches production with no gate. Public order tracking has no rate limit (enumeration risk). **The inventory blocker is mandatory for launch; the status/carrier/preflight gaps are the post-launch order-to-production buildout this product still needs.**

Stale-doc corrections (vs `readme/QA_FINDINGS_2026-06-01.md`)

The 2026-06-01 pass reported two money mismatches with specific dollar figures. Reconciling against current code:

- **Cart → checkout shipping mismatch (\$49 → \$100):** the *symptom* is confirmed and broader than first reported. It is not a single hardcoded \$49 → \$100 jump but a structural divergence with two live causes: (a) the cart previews only the **first active flat-rate method** (`StorefrontCartService.php:102-104`) while checkout lets the customer pick a different (pricier) method with no delta warning; and (b) any **non- flat_rate_ method silently prices to \$0** (`StorefrontCheckoutController.php:488-496`) because carrier rating (EasyPost) was never wired in. So the cart figure and the order's `shipping_cost` can legitimately differ — and for carrier methods the order records \$0 regardless of what was shown. The old “\$49 → \$100” line should be retired in favor of these two concrete code findings. **The mismatch is not fixed.**
- **PDP price mismatch (\$77 vs \$75):** the current findings do not reproduce a fixed \$77-vs-\$75 discrepancy, but they identify the live mechanism that produces PDP ↔ order price drift — the **per-size markup is never applied** because `selected_size_id` is never sent (`ProductPricingCalculator.php:408`). Any product with per-size pricing will show and charge the base price across PDP, cart, and invoice, i.e. the size delta is dropped rather than mis-added. Treat the old “\$77 vs \$75” note as superseded by this root-cause finding.

Readiness recommendations

Clearly labelled recommendations — engineering actions to get the money path to launch-ready, ordered by leverage.

1. **Fix the four blockers first** — inventory decrement on payment-capture (wrapped in a transaction, with oversell guard), real password-reset flow (token + email + reset endpoint), remove the master-OTP backdoor, and stop dropping required file-upload options. Nothing else ships until these are closed.
2. **Add a payment-status-truth test** — assert that offline/cheque orders never display “Payment Successful” while `payment_status='pending'` , and that the confirmation email is gated on `payment_status='paid'` . This pins the largest silent-lie cluster.
3. **Add a shipping round-trip test** — `cart total === order.shipping_cost` for the selected method across flat-rate and carrier methods; this would catch both the method-switch divergence and the carrier-\$0 drop in one assertion.
4. **Add an inventory-decrement test** — two concurrent orders for the last unit must not both succeed; assert stock reaches 0 and the second order is rejected.
5. **Introduce a shared `ErrorState` component** for the storefront and adopt it on `profile/orders/index.vue` , `profile/orders/[id].vue` , `profile/index.vue` , `cart.vue` , and the checkout-config failure path — every money-path view must render loading / empty / **error** (currently several blank or mislabel network errors as “not found”).
6. **Centralize address + payment-method validation** — one shared rule set for `new_shipping_address` / `new_billing_address` used by both order and quote paths, plus a `ValidPaymentMethod` rule scoped to the store; remove the null-coalescing defaults that persist blank addresses.
7. **Run a mobile 375px sweep** on cart and checkout — confirm all quantity, shipping, and payment controls meet the 44×44px tap-target minimum.
8. **Surface gateway-config gaps at save time** — validate/warn on missing `webhook_secret` for webhook gateways and add an Action Center card for pending cheque orders, so non-technical owners aren't left with silently stuck orders.
9. **Scope the order-to-production buildout as the next milestone** — core PRODUCTION/SHIPPED statuses, EasyPost wired into checkout, preflight enabled with a ≥300 DPI gate, print-ready PDF guaranteed (or order rejected) at checkout, and rate-limiting on public order tracking. This is the largest structural gap and should be treated as a tracked post-launch (or launch-gating, per the founder's risk appetite) workflow.

Onboarding & first-run readiness for a brand-new store owner

Source: [readme/research/qa-launch/qa_section_onboarding.md](#)

IN SHORT — KEY TAKEAWAYS

This document checks whether a brand-new store owner can set up their store and take a real order without hitting hidden problems. It found serious issues that could let an owner think everything is ready — only to have checkout fail in front of their first real customer.

The main parts explained simply:

- **The “your store is ready” checklist** — A step-by-step list shown to new owners (like “add a product,” “set up payment”) that ticks green when steps are done. The problem: two of those green ticks are fake — they turn green even when the setup is broken.
- **Fake green: payment gateway** — When an owner enters their payment details (e.g. Stripe keys), the checklist marks it complete without actually testing if those details work. A typo or wrong key still shows green, and the first real order fails with a confusing error.
- **Fake green: shipping** — The checklist marks shipping as complete the moment any shipping option exists — even the example ones added automatically. It never checks if a real, active shipping rate is configured. An owner who has not touched shipping at all still sees a green tick.
- **Checklist disappears after 30 days** — Even if setup is not finished, the checklist banner hides itself after 30 days. A busy store owner who hasn't finished setting up loses their only guide and is left with a half-configured store.
- **Empty first screen** — When a new owner logs in for the first time, they see an empty list with no explanation. Nothing tells them what to do next or what a “store” actually is. It can look like the app is broken.
- **No automatic first store** — The platform does not create a starter store when someone signs up. The owner has to figure out how to create one themselves, with no guidance.
- **No setup wizard** — After creating a store, the owner is dropped onto the main dashboard and left to find each settings page on their own. There is no step-by-step walkthrough (logo → product → payment → shipping → go live).
- **Help text is too technical** — Checklist steps say things like “Connect a payment provider” with a bare link, but don't explain what an API key is, which gateways are supported, or how long it takes. Non-technical owners are left guessing.
- **Demo products aren't clearly labelled** — Sample products are added automatically to show what the catalog looks like, but the banner doesn't make it obvious they are just examples, or that deleting them is permanent.

What to do with this:

Fix the two fake-green steps first — payment and shipping should only show complete when they have actually been tested and are working. Then add a short, guided setup wizard so new owners are never left staring at a blank screen.

Launch-readiness assessment — 2026-06-15 · Audience: founder + engineers · Reviewer perspective: senior QA consultant

Verdict

Not launch-ready for a self-serve, non-technical store owner. The guided-setup scaffolding exists and is thoughtfully built — there is a real “Get your store ready” checklist (`useSetupCheckList.ts`), demo-catalog seeding, and proper empty states with CTAs on list pages. But the **first 10 minutes of a brand-new owner's life on the platform are unguided and have two silent-lie traps in the launch-critical path** (payment + shipping completion checks pass on existence-of-record, not on working-config). Combined, these mean an owner can see an all-green checklist and a “ready” store, then have **checkout fail in front of their first real customer**. There is also no automatic first store on registration and no post-creation wizard, so the very first screen a new tenant sees is an empty store list rather than a “let's get started” flow.

Priorities for launch: fix the two checkout-path silent lies (P0), close the unguided first-run gap (auto-create first store + post-creation wizard), and harden the checklist's auto-expiry so it can't hide an incomplete setup.

Findings

Severity	Finding	Where	Type	Confidence	Recommendation (short)
High	Payment-gateway step marks complete on flags only, never tests credentials — green check while checkout can still fail	<code>useSetupChecklist.ts:247-259</code>	silent-lie	Confirmed in code	Add "Test Connection"; complete only on a successful test, or show "configured but not tested"
High	Shipping step completes on <code>data.length > 0</code> — no check that a rate is active/valid	<code>useSetupChecklist.ts:274-285</code>	silent-lie	Confirmed in code	Require ≥ 1 active rate with a real charge / delivery promise
High	Checklist auto-hides on day 31 regardless of completion — can leave a store stranded mid-setup	<code>useSetupChecklist.ts:135-150</code> (<code>EXPIRE_DAYS=30</code> , line 11)	onboarding-gap	Confirmed in code	Don't expire while launch-critical steps are incomplete; or drop expiry
High	No contextual "what is a store / this is your first login" help on the empty store list	<code>store-management/stores/index.vue</code> ; primitive at <code>store/Select.vue:13</code>	onboarding-gap	Confirmed in code	Add explanatory first-run empty state + "Create Your First Store" CTA
Low	No automatic first-store creation on new-tenant registration — first screen is an empty list	ABSENT (confirmed not in codebase)	onboarding-gap	Confirmed in code	Auto-create a store + seed demo data on tenant create
Medium	No post-creation setup wizard — owner must self-navigate to each settings page	<code>store-management/stores/create.vue</code>	feedback-gap	Likely	Add a 5-minute post-create wizard (logo → product → payment → shipping → preview)
Medium	Checklist step links lack plain-language help for non-technical owners (API key? which gateways?)	<code>useSetupChecklist.ts:67, 89</code>	support-gap	Confirmed in code	Add "Learn more" inline help per step
Medium	"Preview storefront" completion is localStorage-only — resets if browser data cleared	<code>useSetupChecklist.ts:95-109, 211-216, 287-292</code>	field-drop	Confirmed in code	Record server-side, or warn that it's local-only (it is optional)
Medium	Demo-products banner doesn't say deletion is permanent or that items are examples	<code>products/index.vue:26-59</code>	feedback-gap	Confirmed in code	Clarify "examples", call out irreversibility in the banner
Medium	Dashboard widgets load independently with no visible error/timeout fallback shown in template	<code>index.vue:54-62</code>	missing-loading-empty	Likely	Audit each widget for error + timeout end states
Low	Same demo catalog seeded to every store; multi-store tenants see duplicate sample products	<code>StoreOnboardingService.php:809-867</code>	feedback-gap	Likely	Document tenant-global product / per-store pivot model; show store scope in UI

Detail — the launch-critical items

1. The two checkout-path silent lies (P0 — fix before any real customer)

These are the most dangerous onboarding bugs because they violate the platform's own "never surface a fake success" rule precisely at the moment of first revenue.

Payment gateway — `useSetupChecklist.ts:247-259` . Step completion is:

```
step.completed = integrations.some((i: any) => i.is_active && i.is_configured)
```

This only proves a config form was submitted and the flags were set — it does **not** prove the stored API keys are valid, unexpired, or reachable. A non-technical owner who fat-fingers a Stripe secret key, or pastes test keys into a live store, gets a green "Payment gateway configured" check and reasonably concludes they are ready to sell. The first real order then fails with a technical gateway error. **Recommendation:** add a "Test Connection" action on the payment settings page (`/setting/third-party-services`) that does a real round-trip against the stored credentials, and only mark the checklist step complete after a successful test — or, at minimum, distinguish "configured but not tested" from "configured and verified." Re-test (or use a short-TTL cached test result) at checkout so a later credential rotation can't silently break ordering.

Shipping — `useSetupChecklist.ts:274-285` . Completion is simply:

```
step.completed = data.length > 0
```

`StoreOnboardingService` seeds two rates (Standard Shipping and Store Pickup), so this is **green by default on every new store before the owner has looked at it**. It does not check that any rate is active, that any rate carries a real charge or a delivery promise, or that `estimated_days` is set. An owner who deactivates the standard rate, or whose only option is a \$0 rate with no delivery estimate, still sees green — and a customer at checkout is offered "free shipping, no delivery date," which reads as broken. **Recommendation:** require at least one `is_active` rate that has either a positive charge, a free-shipping threshold, or a delivery estimate, and surface the active/inactive count on the shipping settings page.

Both of these are the exact "confirmed-then-discarded / fake-success" bug class this codebase has repeatedly flagged — here it lands on the two steps that gate the ability to take money.

2. Checklist auto-expiry can hide an incomplete setup — `useSetupChecklist.ts:135-150`

`isExpired` (line 135-139) is true once 30 days (`EXPIRE_DAYS`, line 11) have passed since `first_shown_at`, and `shouldShow` (line 142-150) hides the banner when expired:

```
const shouldShow = computed(() =>
  stateLoaded.value && isEligible.value &&
  !state.value.manually_dismissed &&
  !state.value.all_completed &&
  !isExpired.value &&
  !allComplete.value,
)
```

Stale-doc correction: the findings draft described this as an "OR condition." Re-verified — it is actually an AND of negated guards, and `shouldShow` also re-checks live `allComplete.value` alongside the persisted `all_completed` flag. The **behavioral conclusion still holds and is the real problem:** because `!isExpired.value` is one of the ANDed guards, the banner disappears on day 31 *whether or not setup is finished*. A new owner who gets busy for a month — exactly the realistic case for a small print shop — loses their only guidance and is left with a half-configured store and no prompt to finish.

Recommendation: do not expire while any launch-critical (required, non-optional) step is incomplete; reserve expiry, if kept at all, as an escape hatch only once the store is genuinely sellable, and lengthen the window. Note the redundancy between the persisted `all_completed` flag and the live `allComplete` computed — keep one as the source of truth to avoid drift.

3. First-run is unguided: empty list, no auto-store, no wizard

A brand-new tenant's **first screen is an empty store list**, not a welcome flow. The list page does render a proper empty state with an “Add Store” CTA (the shared `store/Select.vue:13` primitive shows “No stores yet - create a store first.”), so this is not a blank-screen bug. The gap is **contextual guidance**: nothing explains that a store is a branded storefront, that creating one is the expected first action, or what gets configured next. To a non-technical owner this can read as “the app is empty/broken.”

Compounding it: **there is no automatic first-store creation on registration** (confirmed absent), and **no post-creation wizard** (`store-management/stores/create.vue` collects name/subdomain/logo/domain then drops the owner onto the dashboard to self-navigate). The pieces to do better already exist — `StoreOnboardingService` can seed demo catalog, FAQs, pages, and a working `WELCOME10` coupon. **Recommendation**: on tenant creation, auto-create a first store named after the owner and run the onboarding seeder, so the very first login lands on a ready-to-customize store with the checklist visible; and add a short post-creation wizard (logo → first product → payment → shipping → preview) that walks the required steps in order with inline help, rather than leaving discovery to the owner.

Detail — secondary items

- **Step help is too technical** (`:67, :89`). “Connect a payment provider” / “Configure shipping rates” with bare links assume the owner knows what a gateway, API key, or shipping rate is. Add per-step “Learn more” cards listing supported gateways, an estimated time, and the one gotcha that matters (“your secret key is private — never share it”).
- **Preview-storefront completion is localStorage-only** (`:95-109, :211-216, :287-292`). The code comment (line 105-108) is candid that it can't be verified server-side. Since the step is `optional` and never blocks auto-hide, this is acceptable — but clearing browser data silently flips the ✓ back off. Either persist a server-side timestamp or, given it's optional, simply note it's a local convenience marker.
- **Demo banner copy** (`products/index.vue:26-59`). The “Remove samples” confirmation does state irreversibility (verified), but the banner itself doesn't, and it invites owners to “customise” samples without clarifying these are illustrative examples, not their real catalog. Tighten the copy to say “examples,” and surface the irreversibility hint before the owner reaches the dialog.
- **Dashboard widgets** (`index.vue:54-62`). Widgets fetch independently and the excerpt shows no error/timeout fallback in the template; with the previously-noted ~10s full load, a slow or 500-ing endpoint could leave a widget skeleton-bound. *Recommendation (labelled)*: audit each `Dashboard*Widget` for explicit error + timeout end states; verify with delayed/500 mocks.
- **Demo catalog per store** (`StoreOnboardingService.php:809-867`). Confirmed: `seedSampleCatalog` is idempotent per store (`StoreProduct::where('store_id', ...)->exists()` guard) and products are created tenant-global via `firstOrCreate` on SKU then attached per store via the pivot. Correct architecturally, but a multi-store tenant sees the same four samples in each store and may not understand the tenant-global / per-store-assignment model. Document the design in-code and show the active store scope on the products page.

Readiness recommendations (labelled — additive, not findings)

1. **Recommendation**: treat “can this store actually take an order?” as a single server-computed readiness signal (payment verified AND ≥1 active valid shipping rate AND ≥1 published product), and drive both the checklist and a storefront “not yet open for orders” guard from it — so the owner-facing state and the customer-facing state can never disagree.
2. **Recommendation**: add a launch smoke path the owner can trigger (“Place a test order”) that exercises payment + shipping end-to-end and reports plain-language pass/fail, converting the two silent lies into an explicit, owner-visible check.
3. **Recommendation**: add lightweight onboarding tests asserting the checklist does **not** report complete when credentials are invalid or no active shipping rate exists — this is the exact silent-lie class the project's own conventions require a

reproducing test for.

Support / Help-Desk Readiness

Source: [readme/research/qa-launch/qa_section_support.md](#)

IN SHORT — KEY TAKEAWAYS

This document checks whether the platform is ready to handle real customer problems before launch. The short answer is: it is not. Right now, when a customer has trouble, their message disappears into a black hole — and when a store owner tries to investigate, they hit a locked door. Both ends of the support loop are broken.

The main parts explained simply:

- **Owners cannot see their own error logs** — When a customer says “checkout is broken,” the store owner has no way to look up what went wrong. The error log page is locked to system administrators only. Owners are left guessing.
- **Contact form submissions go nowhere** — A customer fills out the contact form and gets a vague “Thank you” message. No reference number. No confirmation email. No way to follow up. The message sits in a database that nobody is watching.
- **No ticket system exists** — There is no software to track customer messages, assign them to staff, or measure how fast they are answered. Messages can sit unread indefinitely with no alert.
- **Storefront crashes are silent** — When the store goes down or throws an error, nobody is notified. The error is recorded in a database, but no alarm goes off and no one gets alerted. A store could be broken for hours without the owner knowing.
- **In-app help is almost absent** — The platform has help tooltip components, but they are barely used. The Designer, Forms, and Shipping sections have zero help text. There is no Help menu anywhere.
- **The error page leaves customers stranded** — When a customer hits a crash page, it says “contact support” but gives no email, no phone number, no link, and no error code to quote.
- **Help guides exist but are hidden** — Over 20 help documents have already been written, but they are not linked anywhere in the product. A store owner will never find them.
- **The chat tool is wired wrong** — There is a real-time chat feature, but it is set up for internal staff messaging, not for talking to customers.

What to do with this:

Build the ticket system first — it is the foundation everything else depends on. Then fix the contact form to send customers a reference number and a confirmation email. Without these two things in place, do not promise any support response time to customers.

Assessment date: 2026-06-15 **Scope:** Store-owner (admin) and end-customer (storefront) support paths — in-app help, contact/chat channels, ticketing, FAQ/KB discoverability, and the escalation path for a non-technical owner who gets stuck.

Verdict

Not launch-ready. The platform has no support spine. Customer contact-form submissions are write-only — they land in a `StoreFormSubmission` table that no workflow consumes, with no ticket, no reference number returned to the customer, no confirmation email, and no SLA tracking. The primary users (non-admin store owners) are locked out of their own error logs by an `is_admin` permission wall, so when a customer reports a broken checkout the owner has zero ability to self-diagnose. There is no in-app way for an owner to escalate a problem to Print-Flow support, no dashboard health signal when the storefront throws 500s, and no discoverable help documentation despite 20+ guides sitting offline in `/readme/docs/content/`. Three blockers and five high-severity gaps below must close before real users arrive — each one converts a routine support moment into a dead end for a non-technical owner.

A useful framing for the founder: today, a customer who hits a problem submits into a black box, and an owner who hits a problem has no logs, no ticket queue, and no escalation button. Both ends of the support loop are open.

Prioritized Findings

Severity	Finding	Where	Type	Confidence	Recommendation (summary)
Blocker	Store owners cannot access error logs or diagnostics — total blind spot during incidents	<code>nuxt/app/pages/setting/log/index.vue:230</code> ; <code>app/Http/Controllers/Api/LogController.php:18</code>	support-gap	Confirmed in code	Tenant-scoped diagnostics page for non-admins (own errors + audit only)
Blocker	Storefront contact submissions are silent — no confirmation, reference number, or status tracking	<code>frontstore/app/components/storefront/blocks/DynamicFormBlock.vue:28-36</code> ; <code>StorefrontFormController.php:192-198</code>	feedback-gap	Confirmed in code	Surface submission UUID as reference, confirmation email, public status page
Blocker	No ticket system — submissions never become tickets, no escalation, no SLA	ABSENT — <code>StorefrontFormController.php:163-198</code> stores only	support-gap	Confirmed in code	<code>Ticket</code> model + Support Queue + SLA + action-center alert
High	Storefront 500s not escalated to support staff; owners can't see them	<code>frontstore/.../useErrorReporter.ts:47-61</code> ; <code>StorefrontErrorLogController.php:19-30</code> ; <code>setting/log/index.vue:230</code>	support-gap	Likely	Alert rule on 5xx spike + dashboard health card + owner-scoped visibility
High	In-app help minimal and inconsistent — no help on Designer, Forms, Shipping; no Help menu	<code>nuxt/.../shared/HelpTooltip.vue</code> , <code>HelpIcon.vue</code> ; <code>layouts/store-management.vue:299-354</code>	onboarding-gap	Confirmed in code	Context help on complex pages + global Help menu + first-run tour
High	500 error page has no support contact or error reference	<code>frontstore/app/error.vue:65-68</code>	support-gap	Confirmed in code	Error ID + visible Contact Support + status link
High	Help docs not discoverable from admin UI — 20+ guides exist offline, unlinked	ABSENT — <code>layouts/store-management.vue:299-354</code> ; <code>/readme/docs/content/1.store-owner/</code>	onboarding-gap	Confirmed in code	Add “Help & Docs” sidebar section / header Help menu
High	FloatingChat is internal messaging only — not wired to customer support	<code>nuxt/app/components/chat/FloatingChat.vue:1-100+</code>	support-gap	Likely	Add Support Inbox tab or storefront support-chat widget
Medium	Contact form promises SLA but sends no customer confirmation	<code>frontstore/app/pages/contact.vue:79-88</code> ; <code>StorefrontFormController.php:173-185</code>	feedback-gap	Confirmed in code	Customer confirmation email with reference + status link
Medium	No dashboard health/alert widget — owner gets no heads-up on error spikes or pending messages	ABSENT — <code>nuxt/app/pages/index.vue</code> ; <code>action-center/index.vue</code>	support-gap	Likely	“Support & Health” dashboard card + critical-alert banner
Medium	No in-app way for owners to escalate to Print-Flow support	ABSENT — no Report-to-Support action; <code>LogController</code> has no export/send	support-gap	Likely	“Report Issue to Print-Flow” modal with auto-filled diagnostics

Detail on the most important items

1. (Blocker) Owners are locked out of their own diagnostics

The System Logs page is hard-gated: `nuxt/app/pages/setting/log/index.vue:230` requires `permission: ['is_admin']`, and the backend matches it — `LogController::getErrorLogs()`, `getAuditLogs()`, and `getLogs()` all sit behind `middleware('permission:is_admin')` (`app/Http/Controllers/Api/LogController.php:18`). The primary users of this platform are non-technical, non-admin store owners. When a customer reports “checkout failed,” the owner cannot view the `error_logs` table, file logs, or the audit trail for their own tenant. They are forced to relay a vague description to central support.

Recommendation: Build a store-owner-scoped diagnostics page (e.g. `/setting/errors-my-store`) filtered by `tenant_id`. Show an error-count trend, a recent-errors list (message / timestamp / affected user), and a “Report to Support” button that pre-fills a support request with the error details. Withhold sensitive audit fields (IP, user agent, raw trace) from non-admins, but give them enough to say “this broke May 15 at 2pm for customers in Australia,” plus a “Copy diagnostic code” action. This is the single highest-leverage fix — it unblocks owner self-service and feeds every other support path below.

2. (Blocker) The customer contact form is a black box

On success, `DynamicFormBlock.vue:28-36` shows a generic “Thank You! Your submission has been received.” and an optional redirect. The backend (`StorefrontFormController.php:192-198`) does return the submission `uuid`, but the component extracts only `success_message` and `redirect_url` (line 260) — the UUID is dropped. The customer leaves with no reference number, no email confirmation, and no way to check status or escalate.

Recommendation: Display the UUID as a human reference (e.g. `Reference: PF-20260615-a1b2c3d4`), send an automated confirmation email with that code and an SLA promise, and expose a public, no-auth `GET /contact/submission/:uuid` status page (submitted / read / responded) backed by `first_read_at` / `responded_at` columns. Pairs directly with finding #9 (the contact page already *promises* a one-business-day reply it never confirms).

3. (Blocker) There is no ticketing workflow at all

`StorefrontFormController::submit()` persists the row and fires `notify_emails` to the admin if configured (lines 173-185) — and stops. No `Ticket` model, `TicketController`, support queue, assignment, or SLA exists anywhere in the codebase. The submission is write-only; nothing consumes it. Owners see no “pending submissions” count on their dashboard, so messages can sit unread indefinitely.

Recommendation: Introduce a `Ticket` model (`uuid`, `form_submission_id` FK, `tenant_id`, `subject`, `priority`, `assigned_to`, `status`, `first_response_at`, `resolved_at`, `sla_due_at`). On submit: create the ticket, push to a `/support/tickets` queue, raise an action-center “Pending Submissions” alert, and optionally auto-assign. Surface SLA compliance (% replied within 24h). This is the backbone that findings #2, #4, #8, #9, and #10 all hang off — build it first.

4. (High) Storefront 5xx errors reach the DB but never reach a human

The storefront auto-reports 5xx errors: `frontstore/error.vue:104-114` calls `useErrorReporter` (`useErrorReporter.ts:47-61`), which POSTs to `/api/v1/storefront/errors`; `StorefrontErrorLogController.php:19-30` tenant-scopes and persists them. But nothing fires an alert, owners can't see them (the `is_admin` wall from #1), and they're discoverable only via the admin-only log API. A storefront could be down for hours with no one notified.

Recommendation (rule, confidence: likely): When `status_code >= 500 AND source=frontend AND count > 2` in a 5-minute window, raise an action-center alert, notify assigned support contacts, and show a “System Health” card with the live error rate. Add an `ErrorLogAlert` dedupe record. Give owners a tenant-scoped 5xx count + last-error timestamp on their diagnostics page (#1).

5–7. (High) Help and escalation are invisible from the product

- **In-app help is thin and lopsided.** `HelpTooltip.vue` / `HelpIcon.vue` support rich popovers (title, content, examples, external link) and are used in ~81 places — but heavily concentrated in pricing-rules, with **zero** coverage in Designer, Forms, or Shipping, and no instance passes a `link` prop despite the components supporting external doc links. There is no global Help menu (`layouts/store-management.vue:299-354` defines six nav groups, none for Help).
- **The 500 error page strands the customer.** `frontstore/app/error.vue:65-68` says “please contact support” with no email, phone, link, or error reference. The customer cannot escalate without manually finding `/contact`.
- **Docs exist but are unreachable.** 20+ guides live in `/readme/docs/content/1.store-owner/` and are referenced nowhere in `nuxt/app/`. A non-technical owner will never find them.

Recommendations: Add context-sensitive help to the complex pages (Designer canvas, form field types, shipping rule logic, pricing conditions); add a global Help menu (Help Center link, “What’s this?” inline-help toggle, Contact Support); put a reference code + visible Contact Support button on the 500 page; and add a “Help & Docs” sidebar section that surfaces the existing guides. Consider a first-7-days guided tour on Designer / Forms / Shipping.

8. (High) The chat widget is the wrong chat

`FloatingChat.vue` is a complete real-time messaging UI (FAB, thread sidebar, unread badges, sockets) but its own comment (line 3) scopes it to internal team messaging. It is not connected to customer tickets, has no storefront-facing widget, and no assignment/SLA model. The infrastructure for support chat largely exists — it just isn’t pointed at customers.

Recommendation (confidence: likely): Add a “Support Inbox” tab that renders tickets (from #3) as chat threads, and/or a public storefront support widget that lets customers message support directly. Link the two so staff replies reach the customer in real time.

9–11. (Medium) Confirmation, visibility, and escalation finishing touches

- **Contact page over-promises** (`contact.vue:79-88`). It states “We’ll get back to you within one business day,” but `notify_emails` go only to the admin (`StorefrontFormController.php:173-185`) — the customer gets no confirmation. Close this together with #2.
- **No dashboard health widget.** `nuxt/app/pages/index.vue` has no error-rate, pending-tickets, or SLA card; the Action Center is internal-only (domain/payment alerts, not support/error alerts). An owner with 10 pending messages and 5 storefront errors sees nothing — and can’t reach the logs anyway (#1).
- **No owner-to-Print-Flow escalation.** There is no “Report Issue” action in the admin and no way to bundle diagnostics. Owners fall back to external email/phone, losing all context.

Recommendations: Send a customer confirmation email with reference + status link; add a “Support & Health” dashboard card (error counts, pending tickets, SLA %, service status) plus a critical-alert banner on 5xx spikes; and add a “Report Issue to Print-Flow Support” modal that auto-fills `tenant_id`, recent error logs, current URL, and user agent into a `SupportRequest` record.

Stale-doc corrections

No prior support/help-desk findings were on record to re-verify; all items above are first-pass and code-grounded against the current `store-theme-change` branch. Treat this section as the baseline for future re-verification.

Readiness recommendations (clearly labelled — not findings)

1. **Recommendation:** Sequence the three blockers as one epic — the `Ticket` model (#3) is the dependency for confirmation/status (#2, #9) and owner visibility (#1, #10). Ship that backbone first.

2. **Recommendation:** Define and display an SLA from day one (e.g. "within 1 business day") only once a confirmation + status path exists — promising it without delivering (today's `contact.vue` state) erodes trust faster than no promise.
3. **Recommendation:** Add a single public status/health endpoint (`GET /api/v1/storefront/errors/:id` → `{status, message, resolved_at}`) that both the 500 page and the contact status page can consume, so customers can self-check recovery.
4. **Recommendation:** Wire the offline `/readme/docs/content` guides into the product before launch even as plain external links — undiscoverable docs are equivalent to no docs for a non-technical owner.

Real-User QA, Onboarding & Support Readiness — Print-Flow-360

Source: [readme/research/qa-launch/REAL_USER_QA_READINESS_2026-06-15.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a deep check of whether Print-Flow-360 is ready for real store owners and their customers. Researchers read through the actual code — not the documentation — and found 107 problems, including 8 that must be fixed before anyone real uses the platform. The short answer is: not ready yet, but the path to ready is clear.

The main parts explained simply:

- **The overall verdict** — The platform can show products, build a cart, and place an order. But the money trail has gaps (amounts shown to the customer do not always match what gets charged), there is no way for store owners to get help when something breaks, and the platform has no way to learn when something goes wrong after a sale.
- **The 8 must-fix blockers** — Eight problems so serious they must be solved before any real user touches the platform. Two are security holes: a secret backdoor code (`702702`) lets anyone log into any admin account, and the “forgot password” button for customers looks like it works but never actually sends an email — a locked-out customer stays locked out forever.
- **The money path** — The journey from signing up → choosing a product → cart → checkout → payment → getting the order shipped. Several points leak: stock is never reduced when an order is placed (so the same item can be sold to unlimited people), the tax shown in the cart can differ from the tax at checkout, and offline payments (cheque, bank transfer) show “Payment Successful” even though money has not arrived.
- **Onboarding / first-run** — What a brand-new store owner sees when they log in for the first time. Right now they land on a blank screen with no guidance. The setup checklist shows steps as “done” even when they are not properly set up (for example, marking the payment gateway as ready without checking that the credentials actually work).
- **Support and help-desk** — What happens when something breaks. Currently: nothing useful. Store owners cannot see their own error logs. When a customer fills in a contact form, no reference number is given and no confirmation email is sent. There is no ticket system — messages are stored but never followed up.
- **Feedback capture** — How the platform learns that a customer was unhappy or that a print job was poor. Right now it cannot. There is no post-delivery satisfaction survey, no way to detect when a checkout failed versus when a customer simply left, and no abandoned-cart recovery.

What to do with this:

Fix the two security holes first (the backdoor login code and the broken password reset) — these are urgent regardless of launch timing. Then work through the remaining six blockers in order, focusing on making sure money totals match end-to-end and that stock actually decrements when an order is paid.

Date: 2026-06-15 **Scope:** Pre-launch readiness for a platform that has **not yet been used by real users**, whose primary users are **non-technical store owners** who will not file good bug reports. **Method:** 13 read-only code-mapping sweeps + adversarial QA probes against **current code** (not docs), then synthesis. 26 agents, ~2.0M tokens. Every finding cites `file:line` and a confidence rating; runtime-only suspicions are marked `needs-runtime-check`. **Why this doc exists:** the older `readme/QA_*`, `PRE_LAUNCH_PLAN.md`, and `PLATFORM_GAP_ASSESSMENT_2026-06-07.md` are explicitly flagged stale in `GOAL.md`. This is a fresh re-verification plus deep coverage of three areas none of those docs really tackled: **first-run onboarding, support/help-desk, and feedback capture**.

0. TL;DR — Verdict

Not launch-ready for self-serve, non-technical store owners. The platform can browse, configure, cart, and place an order — but the money path leaks in ways the user is never told about, there is **no support spine**, and there is **no way to learn from real users** once they hit a problem.

107 findings: 8 blockers · 36 high · 40 medium · 23 low.

The single most important pattern, again, is the house “**silent-lie**” class (25 of 107 findings): the UI says *Saved / Paid / Successful / link sent* while the write silently failed or never happened. Two are outright **security backdoors**, both re-verified by hand in current code:

- A **hardcoded master OTP** `702702` logs in as *any* admin if the `MASTER_OTP` env var is unset (`AuthController.php:368`).
- **Customer password reset is a stub** that logs a TODO and returns “*you will receive a password reset link shortly*” — no email is ever sent (`CustomerAuthController.php:496-501`). A locked-out customer is permanently locked out and told help is on the way.

Readiness scorecard

Area					Total	Verdict
Money path (signup → ... → fulfillment)	4	19	22	14	59	Not ready — money moves/misreports + 2 auth backdoors
Loading/empty/error states + mobile	0	5	6	1	12	Critical pages blank on failure; no shared ErrorState
Onboarding / first-run	0	4	5	2	11	Unguided first-run; setup checklist gives false “done”
Support / help-desk	3	5	3	0	11	No support spine at all (no tickets, no owner diagnostics)
Feedback capture	1	3	4	6	14	Platform can transact but cannot listen

Section deep-dives (companion files)

- `readme/qa_section_money_paths.md` — the 7-stage money path, stage by stage.
- `readme/qa_section_onboarding.md` — brand-new store-owner first-run experience.
- `readme/qa_section_support.md` — store-owner + customer support readiness.
- `readme/qa_section_feedback.md` — reviews, surveys/NPS, proactive problem detection.

1. The 8 blockers (fix before any real user touches this)

#	Blocker	Area	Where	Class
1	Hardcoded master OTP 702702 bypasses admin 2FA if <code>MASTER_OTP</code> env unset	Auth	<code>app/Http/Controllers/AuthController.php:368</code>	security / silent-lie
2	Customer password reset is non-functional — returns “link sent”, sends nothing (TODO stub)	Auth	<code>app/Http/Controllers/Api/Storefront/CustomerAuthController.php:496-501</code>	silent-lie
3	Stock is never decremented at order placement or payment → unlimited oversell	Fulfillment	<code>StorefrontCheckoutController.php</code> (no <code>deductStock</code>); <code>UpdatePaymentAndOrder.php:155-166</code>	money-mismatch
4	Required file-upload product options silently dropped from validation → buyer orders a print job with no artwork, UI says OK	PDP	<code>frontstore/app/stores/productInfo.ts:139-146, 709-714</code>	silent-lie
5	Store owners cannot see their own error logs (gated to <code>is_admin</code>) → blind during incidents	Support	<code>nuxt/.../setting/log/index.vue:230</code> ; <code>LogController.php:18</code>	support-gap
6	Contact-form submissions are silent — no reference number, no confirmation email, no status	Support	<code>DynamicFormBlock.vue:28-36</code> ; <code>StorefrontFormController.php:192-198</code>	silent-lie
7	No support ticket system exists — submissions are write-only, no escalation/SLA	Support	ABSENT (no <code>Ticket</code> model/controller; only <code>StoreFormSubmission</code>)	support-gap
8	No post-delivery satisfaction/NPS loop — platform cannot learn it shipped a bad job	Feedback	ABSENT (no survey model/job/template)	feedback-gap

 **Blockers 1 & 2 are security/credential issues** — they should be treated as the top priority regardless of launch timing. Both were re-verified by hand in current code for this report.

2. Critical money path — stage by stage

Full detail and the complete 73-row deduped findings table are in `readme/qa_section_money_paths.md`. Summary of where each of the 7 stages stands:

Stage	State	Headline issues
1. Signup & Auth	●	Master-OTP backdoor; dead password reset; OTP-resend endpoints unthrottled.
2. Product config / PDP / pricing	●	Pricing <i>math</i> is largely sound (good news), but required file-upload options dropped from validation (●), and per-size pricing never applied to base price (money-mismatch).
3. Cart	●	Cart ↔ checkout tax formula differs (cart store-wide rate vs checkout per-item); cart shows first flat-rate method, checkout lets customer pick another; guest cart key not tenant-scoped (localStorage collision on shared domains).
4. Checkout	●	New shipping/billing address arrays have no FormRequest validation → fields silently set to empty string; <code>CheckoutCustomFieldsTest</code> failing (possible live regression); shipping cost can resolve to \$0 for any non- <code>flat_rate_</code> method.
5. Payment & gateways	●	Offline (cheque/bank/pickup) orders show “Payment Successful” while <code>payment_status='pending'</code> ; missing <code>webhook_secret</code> hangs orders in pending forever; cheque orders never auto-reconcile.
6. Order creation & confirmation	●	Confirmation email sent at draft (before payment) saying “received and being processed”; email shows item subtotal only , not the real total with shipping/tax/coupons.
7. Fulfillment / production / shipping	●	Structurally incomplete: no PRODUCTION/FULFILLED/SHIPPED statuses , EasyPost integration built but never wired into checkout, preflight validation disabled, print-ready PDF can be NULL at checkout (blocks later production download), no partial fulfillment, tracking entered by hand. The “paid → shipped” spine does not exist.

Money-math truth (the launch gate): the platform does **not** currently guarantee `cart total === checkout total === order total === invoice`, and inventory does not decrement. The cart/checkout **tax** and **shipping-method** divergence and the **\$0-shipping** path are the live money-mismatch risks; the older “\$49 → \$100 shipping” symptom from `QA_FINDINGS_2026-06-01.md` is superseded by these (see §6).

3. Onboarding & first-run readiness

Full detail in `readme/qa_section_onboarding.md`. Verdict: ● **a brand-new owner is not guided to a working store.**

- **No guided first-run.** A freshly registered tenant lands on an empty store list with **no auto-created first store** and **no post-registration wizard** (`store/Select.vue:13` lacks an empty state).
- **The setup checklist gives false “done” signals (silent-lie class).** The payment-gateway step marks complete on `is_active / is_configured` flags **without testing credentials** (`useSetupChecklist.ts:247-259`); the shipping step completes on `data.length > 0` **without checking any rate is active/valid** (`:274-285`) — and `StoreOnboardingService` seeds 2 rates, so it shows green by default.
- **The checklist auto-expires at day 31 regardless of completion** (`useSetupChecklist.ts:135-150` , `EXPIRE_DAYS=30`) — a half-configured store loses its only guidance.
- Empty states across admin mostly exist with CTAs, but lack contextual help on the hard features (Designer, Form Builder, Pricing Rules, Shipping).

Key recommendation: a server-computed “can this store take a real order?” signal (payment gateway verified + ≥1 published product + valid shipping + store info) that drives both the checklist and a “you’re ready to sell” state — replacing the current existence-only flag checks.

4. Support / help-desk readiness

Full detail in `readme/qa_section_support.md`. Verdict: ● **there is no support spine.**

- **Owner blind spot (🔴):** non-admin store owners are walled out of their own error logs (`setting/log/index.vue:230` , `LogController.php:18`). When their storefront 500s, they cannot see why and cannot self-diagnose.
- **Silent contact form (🔴):** customer submissions return no reference number, no confirmation email, no status — `DynamicFormBlock.vue` even **drops the returned UUID**.
- **No ticketing (🔴):** submissions write to `StoreFormSubmission` and stop. No `Ticket` model, no escalation, no SLA, no reply panel.
- **High gaps:** storefront 500s never alert anyone (errors persist to `error_logs` but only admins ever look); no global Help/Docs/Support menu in admin (`store-management.vue:299-354`); the 500 error page has **no contact info or error reference** (`frontstore/app/error.vue:65-68`); ~20 store-owner help docs exist in `readme/docs/content/` but are **not linked anywhere in-product**; `FloatingChat` is internal-messaging only, not wired to customer support.

Key recommendation: the cheapest high-impact win is **surfacing what already exists** — link the offline docs into a Help menu, show contact details + an error reference on error pages, and give owners a tenant-scoped diagnostics view — before building a full ticket system.

5. Feedback capture readiness

Full detail in `readme/qa_section_feedback.md` . Verdict: 🔴 **the platform can transact but cannot listen**.

- **No post-delivery satisfaction/NPS loop (🔴):** `EmailTemplateEnum.php` has 15 templates, none for feedback; no `OrderFeedback` model, no `OrderDelivered` listener. The platform cannot learn it shipped a bad job.
- **Checkout/payment failures are invisible as friction (🟡):** `useGtm.ts` fires only happy-path events; `ErrorLogService` excludes `ValidationException` (`:35`), so a 422 checkout failure looks identical to a cart abandon. You cannot tell “they left” from “we broke.”
- **No abandoned-cart recovery (🟡) and vanity analytics (🟡)** that answer no friction question.
- **“Owner is never told” cluster (🟡):** pending reviews never notify the owner; inbound order messages have a storefront UI but no admin reply panel (`needs-runtime-check`); error spikes never alert.
- Note (\$0 violation): raw `SQLSTATE` /stack traces are shown to non-technical owners in `setting/log/index.vue:71,93-97` — and the mapping must target **PostgreSQL** `SQLSTATE` codes (the platform is Postgres, not MySQL).

Key recommendation: the transactional email plumbing (`ORDER_SHIPPED` , `PAYMENT_FAILED` templates already exist) is the cheapest insertion point for a 1-click NPS survey and abandoned-cart recovery — the lift is smaller than previously assumed.

6. Stale-doc corrections (vs. earlier QA docs)

`GOAL.md` is right that the old docs are stale. Reconciled against current code:

- **Cart → checkout shipping “\$49 → \$100”** (`QA_FINDINGS_2026-06-01.md` §1): the *specific* symptom is not the live bug anymore — checkout no longer trusts client totals. **However**, the current live shipping risks are different and real: shipping resolves to **\$0** for any non- `flat_rate_` method, and the cart shows the *first* flat-rate method while checkout lets the customer pick a *different* one. Treat the shipping path as **still not parity-safe**, just for new reasons.
- **PDP “\$77 vs \$75” price mismatch:** the pricing math is now largely sound; the live PDP money issue is **per-size pricing never applied to base price** (`productInfo.ts:940-946`) — a different mismatch than the old report.
- **`ORDER_SHIPPED` / `PAYMENT_FAILED` templates:** confirmed to **exist** — they are transactional, not feedback templates, but they’re the recommended hook for the missing survey/recovery flows.
- **Setup-checklist `shouldShow` logic:** re-verified — it is an **AND of negated guards** (`useSetupChecklist.ts:142-150`), not the “OR condition” an early draft claimed. Behavioral conclusion (auto-hide at day 31) is unchanged.

7. Recommended launch gate + QA process

Hard launch gate (must be green):

1. Remove the master-OTP backdoor (#1); ship a real customer password reset (#2). (*security*)
2. Decrement inventory on payment-captured, atomically (#3); add a test.
3. Stop showing “Payment Successful” for unpaid offline orders; the success page must read the real `payment_status`.
4. Validate new address arrays + required file-upload options in their FormRequests (no silent empty-string / dropped artwork).
5. Money-truth test suite: `cart total === checkout === order === invoice` incl. shipping + tax, plus a shipping round-trip test (no \$0 path).
6. A shared **ErrorState** component on the critical money-path pages (checkout, orders list, order detail) — no page blanks on fetch failure.

Then (pre-public, churn-preventers): server-computed “ready to sell” onboarding signal; surface existing help docs + error references + owner diagnostics (support spine v1); 1-click post-delivery NPS + abandoned-cart recovery on the existing email plumbing.

Process going forward (non-technical users won't report bugs — you must catch these yourself):

- Run `readme/QA_TEST_CHECKLIST.md` on desktop **and at 375px**, focusing on the all-three-states rule (loading/empty/error) and the silent-lie class.
- Add a regression test for every blocker/high in §1–§2 — a finding with no test is not fixed.
- Wire proactive detection (error-spike alerting + checkout-failure events) so the next class of bugs surfaces from telemetry, not from a churned customer.

Generated from 13 code-grounded research streams on 2026-06-15. Section files: `qa_section_money_paths.md`, `qa_section_onboarding.md`, `qa_section_support.md`, `qa_section_feedback.md`. Raw findings (107) available in the workflow output.